Predictive analytics

Block 2 - Multiple Linear Regression

Ca' Foscari University of Venice

Academic year 2024/2025

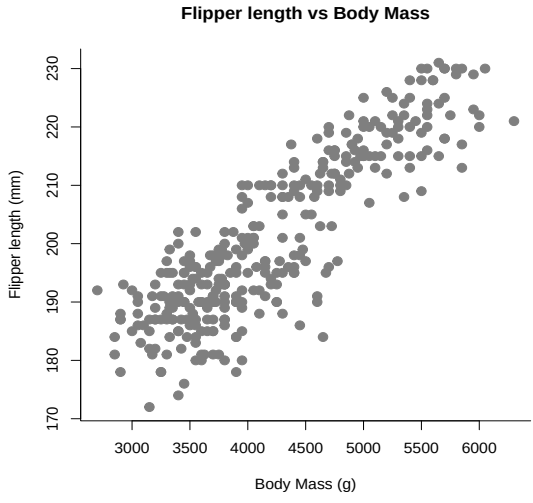
Multiple Linear Regression [1]

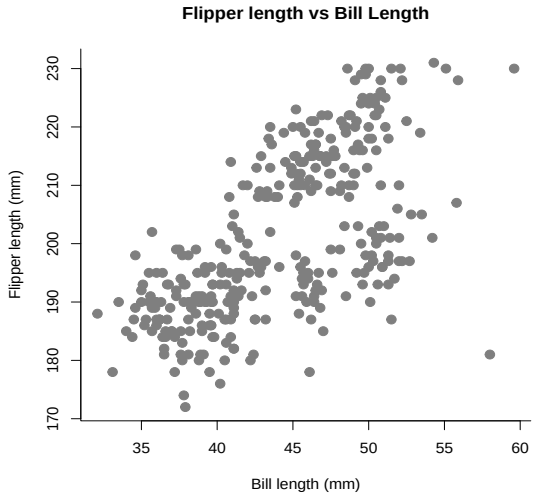
[1] Material in these slides was heavily influenced by David Dalpiaz *Applied Statistics with R!*, the book is under active development.

Multiple predictors

- It is rarely the case that a response variable will only depend on a single variable
- For example, flipper length might be related to both body mass and bill length

Multiple predictors





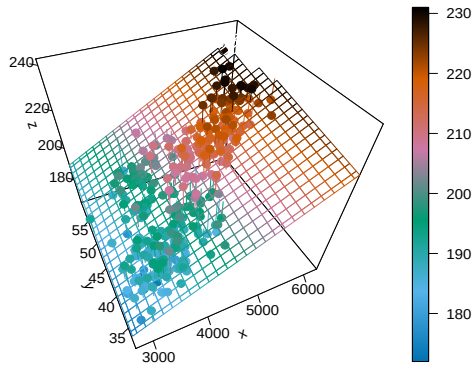
Multiple predictors

- We can use a linear model:

$$Y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \varepsilon_i, \qquad i = 1, 2, \ldots, n$$

  where $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$ (and independent).
- In this notation we will define:
  - x_{i1} as the body mass of the ith penguin.
  - x_{i2} as the bill length of the ith penguin.
  - Extend the least square idea to higher dimension
  - Now $\widehat{MSE}$ depends on $(\beta_0, \beta_1, \beta_2)$
- The data points (x_{i1}, x_{i2}, y_i) now exist in 3-dimensional space, so instead of fitting a line to the data, we will fit a plane

Multiple predictors



Multiple predictors

- How do we find such a plane?
- We would like to minimize

$$f(\beta_0, \beta_1, \beta_2) = \sum_{i=1}^{n} (y_i - (\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2}))^2$$

  with respect to β_0, β_1, and β_2.
- We take a derivative with respect to each of β_0, β_1, and β_2 and set them equal to zero, then solve the resulting system of equations.

$$\begin{cases} \dfrac{\partial f}{\partial \beta_0} = 0 \\ \dfrac{\partial f}{\partial \beta_1} = 0 \\ \dfrac{\partial f}{\partial \beta_2} = 0 \end{cases}$$

Multiple predictors

- We obtain the

Estimating Equations

$$n\beta_0 + \beta_1 \sum_{i=1}^{n} x_{i1} + \beta_2 \sum_{i=1}^{n} x_{i2} = \sum_{i=1}^{n} y_i$$

$$\beta_0 \sum_{i=1}^{n} x_{i1} + \beta_1 \sum_{i=1}^{n} x_{i1}^2 + \beta_2 \sum_{i=1}^{n} x_{i1}x_{i2} = \sum_{i=1}^{n} x_{i1}y_i$$

$$\beta_0 \sum_{i=1}^{n} x_{i2} + \beta_1 \sum_{i=1}^{n} x_{i1}x_{i2} + \beta_2 \sum_{i=1}^{n} x_{i2}^2 = \sum_{i=1}^{n} x_{i2}y_i$$

- (Note: from the definition of β_0 we can see that the plane will pass through $(\overline{x}_1, \overline{x}_2, \overline{y})$).

Multiple predictors

- We let R solve for us:

```
pfit <- lm(flipper_length_mm~body_mass_g+bill_length_mm, data = penguins)
coef(pfit)
```

```
  (Intercept)    body_mass_g bill_length_mm
122.26867040     0.01301733     0.54403527
```

$$\hat{y} = 122.27 + 0.01x_1 + 0.54x_2$$

- β_2 represent the change in flipper length (y) for two penguins with identical body mass (x_1) which only differ by one unit of the bill length (x_2) variable.
- The model is **additive**.

Matrix approach to regression

- For an arbitrary number $p-1$ of predictor variables we can consider the (additive and linear) model

$$Y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_{p-1} x_{i(p-1)} + \varepsilon_i, \qquad i = 1, 2, \ldots, n$$

  where $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$.
  - there are $p-1$ predictor variables, $x_1, x_2, \cdots, x_{p-1}$.
  - There are a total of p β-parameters and a single parameter σ^2 for the variance of the errors

Matrix approach to regression

Stack together the n linear equations that represent each Y_i in a column vector, we get:

$$\begin{bmatrix} Y_1 \\ Y_2 \\ \vdots \\ Y_n \end{bmatrix} = \begin{bmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1(p-1)} \\ 1 & x_{21} & x_{22} & \cdots & x_{2(p-1)} \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_{n1} & x_{n2} & \cdots & x_{n(p-1)} \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \vdots \\ \beta_{p-1} \end{bmatrix} + \begin{bmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \vdots \\ \varepsilon_n \end{bmatrix}$$

$$Y = X\beta + \varepsilon$$

$$Y = \begin{bmatrix} Y_1 \\ Y_2 \\ \vdots \\ Y_n \end{bmatrix}, \quad X = \begin{bmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1(p-1)} \\ 1 & x_{21} & x_{22} & \cdots & x_{2(p-1)} \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_{n1} & x_{n2} & \cdots & x_{n(p-1)} \end{bmatrix} \quad \beta = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \vdots \\ \beta_{p-1} \end{bmatrix}, \quad \varepsilon = \begin{bmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \vdots \\ \varepsilon_n \end{bmatrix}$$

Matrix approach to regression

- With the observed data (the realization of Y):

$$y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

we can estimate β by minimizing

$$f(\beta_0, \beta_1, \beta_2, \cdots, \beta_{p-1}) = \sum_{i=1}^{n} (y_i - (\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_{p-1} x_{i(p-1)}))^2,$$

which would require taking p derivatives, which result in the following

Matrix approach to regression

Estimating Equations

$$\begin{bmatrix} n & \sum_{i=1}^n x_{i1} & \cdots & \sum_{i=1}^n x_{i(p-1)} \\ \sum_{i=1}^n x_{i1} & \sum_{i=1}^n x_{i1}^2 & \cdots & \sum_{i=1}^n x_{i1}x_{i(p-1)} \\ \vdots & \vdots & & \vdots \\ \sum_{i=1}^n x_{i(p-1)} & \sum_{i=1}^n x_{i(p-1)}x_{i1} & \cdots & \sum_{i=1}^n x_{i(p-1)}^2 \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_{p-1} \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^n y_i \\ \sum_{i=1}^n x_{i1}y_i \\ \vdots \\ \sum_{i=1}^n x_{i(p-1)}y_i \end{bmatrix}$$

- The estimating equations can be written in matrix notation,

$$X^\top X\beta = X^\top y.$$

- We can then solve this expression by multiplying both sides by the inverse of $X^\top X$, which exists, provided the columns of X are linearly independent. We obtain estimates of β as:

$$\hat{\beta} = \left(X^\top X\right)^{-1} X^\top y$$

Matrix approach to regression

- In R, we create an X matrix. Note that the first column is all 1s, and the remaining columns contain the data.

```
n <- nrow(penguins)
X <- cbind(rep(1, n), penguins$body_mass_g, penguins$bill_length_mm)
p <- ncol(X)
y <- penguins$flipper_length_mm
beta.hat <- solve(t(X) %*% X) %*% t(X) %*% y; t(beta.hat)
```

```
        [,1]       [,2]      [,3]
[1,] 122.2687 0.01301733 0.5440353
```

```
# same value
coef(pfit)
```

```
  (Intercept)    body_mass_g bill_length_mm
 122.26867040     0.01301733     0.54403527
```

Matrix approach to regression

- Actually R builds the X matrix from the formula (and then does some more optimised linear algebra to obtain the estimates):

```
head(model.matrix(pfit))
```

	(Intercept)	body_mass_g	bill_length_mm
1	1	3750	39.1
2	1	3800	39.5
3	1	3250	40.3
5	1	3450	36.7
6	1	3650	39.3
7	1	3625	38.9

```
head(X)
```

	[,1]	[,2]	[,3]
[1,]	1	3750	39.1
[2,]	1	3800	39.5
[3,]	1	3250	40.3
[4,]	1	3450	36.7
[5,]	1	3650	39.3
[6,]	1	3625	38.9

- There is also a ‘fit$model‘ object - what do you think it is?

Matrix approach to regression

- In our new notation, the predicted values can be written

$$\hat{y} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_n \end{bmatrix} = X\hat{\beta} (= X \left(X^\top X \right)^{-1} X^\top y),$$

  while the vector for the residual values is

$$e = \begin{bmatrix} e_1 \\ e_2 \\ \vdots \\ e_n \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} - \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_n \end{bmatrix}.$$

Matrix approach to regression

- The estimate for σ^2:

$$s_e^2 = \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{n-p} = \frac{e^\top e}{n-p}$$

  Recall, we like this estimate because it is unbiased, that is,

$$\mathrm{E}[S_e^2] = \sigma^2$$

- Note that the change from the SLR estimate to now is in the denominator. Specifically we now divide by $n-p$ instead of $n-2$.
- (Or actually, we should note that in the case of SLR, there are two β parameters and thus $p = 2$.)

Matrix approach to regression

- Also note that if we fit the model $Y_i = \beta + \varepsilon_i$ that $\hat{y} = \bar{y}$ and $p = 1$ and s_e^2 would become

$$s_e^2 = \frac{\sum_{i=1}^{n}(y_i - \bar{y})^2}{n-1}$$

  which is likely to be the very first definition of sample standard deviation you saw in the previous statistics courses.
- In this case we only fit one parameter β, so we lose one degree of freedom and divide by $n-1$. In general, we are estimating p parameters, the β parameters, so we lose p degrees of freedom.
- Typically we are interested in s_e (which R calls the residual standard error)

$$s_e = \sqrt{\frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{n-p}}.$$

Matrix approach to regression

- In R, we can directly access s_e for a fitted model, as before:

```
summary(pfit)$sigma
```

```
[1] 6.419285
```

- We can now verify that our math above is indeed calculating the same quantities.

```
y_hat = X %*% solve(t(X) %*% X) %*% t(X) %*% y
e     = y - y_hat
sqrt(t(e) %*% e / (n - p))
```

```
         [,1]
[1,] 6.419285
```

```
sqrt(sum((y - y_hat) ^ 2) / (n - p))
```

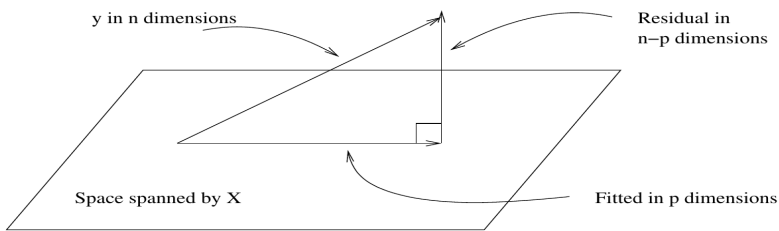
```
[1] 6.419285
```

Geometrical interpretation

- For n observations we estimate β by minimizing,

$$\sum_{i=1}^{n}(y_i - (\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_{p-1} x_{i(p-1)}))^2 = (y - X\beta)^\top (y - X\beta)$$

- Geometrically speaking, $y \in \mathbb{R}^n$ and the p columns of X are also vectors of $\mathbb{R}^n$



Geometrical interpretation

- The problem is to find β such that vector $X\beta$ (i.e. a linear combination of the column vectors in X) is close to y.
- The solution

$$\hat{y} = X\hat{\beta} = X(X^\top X)^{-1}X^\top y = Hy$$

where

$$H = X(X^\top X)^{-1}X^\top$$

is the **orthogonal projection matrix**.

- The conceptual purpose of the model is to represent, as accurately as possible, something complex - y which is n-dimensional - in terms of something much simpler - the model which is p-dimensional.

Geometrical interpretation

- If our model is successful, the structure in the data should be captured in those p dimensions, leaving just random variation in the residuals which lie in an $n - p$ dimensional space.

$$\begin{array}{rcccc} \text{Data} & = & \text{Systematic structure} & + & \text{Random variation} \\ n \text{ dimensions} & = & p \text{ dimensions} & + & n - p \text{ dimensions} \end{array}$$

Multivariate Gaussian distribution

- We will consider a random vector X of n random variables i.e. $X = (X_1, X_2, \ldots, X_n)^\top$
- A random vector is said to be n-variate Gaussian distributed if every linear combination of its n components has a univariate Gaussian distribution.
- The multivariate Gaussian distribution of a n-dimensional random vector can be written in the following notation: $X \sim \mathcal{N}_n(\mu, \Sigma)$ with

$$\mu = \begin{bmatrix} \mu_1 \\ \vdots \\ \mu_n \end{bmatrix} \qquad \Sigma = \begin{bmatrix} \sigma_{11} & \sigma_{12} & \cdots & \sigma_{1n} \\ \sigma_{21} & \sigma_{22} & \cdots & \sigma_{2n} \\ \vdots & \vdots & \ddots & \\ \sigma_{n1} & \sigma_{n2} & \cdots & \sigma_{nn} \end{bmatrix}.$$

where $\mu_j = \mathbb{E}[X_j]$, $\sigma_{ij} = \mathrm{Cov}[X_i, X_j]$

Multivariate Gaussian distribution

- The multivariate normal distribution is said to be "non-degenerate" when the symmetric covariance matrix Σ is positive definite.
- In this case the distribution has density

$$f_X(x) = (2\pi)^{n/2}|\Sigma|^{-1/2}\exp\left\{-\frac{1}{2}(x-\mu)^\top\Sigma^{-1}(x-\mu)\right\}$$

- If $Y = AX + b$ is an **affine transformation** of $X \sim \mathcal{N}(\mu, \Sigma)$ where b is an $m \times 1$ vector of constants and A is a constant $m \times n$ matrix, then Y has a multivariate Gaussian distribution with expected value $A\mu + b$ and variance-covariance matrix $A\Sigma A^\top$ i.e., $Y \sim \mathcal{N}_m(A\mu + b, A\Sigma A^\top)$
- In particular, any subset of the Y_i has a marginal distribution that is also multivariate normal.

Sampling distribution for $\hat{\beta}$

- We want to obtain the distribution of the $\hat{\beta}$ vector

$$\hat{\beta} = \begin{bmatrix} \hat{\beta}_0 \\ \hat{\beta}_1 \\ \hat{\beta}_2 \\ \vdots \\ \hat{\beta}_{p-1} \end{bmatrix}$$

- We consider $\hat{\beta}$ to be a random vector, thus we use Y instead of the data vector y:

$$\hat{\beta} = \left(X^\top X\right)^{-1} X^\top Y$$

- As for SLR we assume that Y is (conditionally) normal

Sampling distribution for $\hat{\beta}$

- We can prove that

$$\hat{\beta} \sim \mathcal{N}_p\left(\beta, \sigma^2\left(X^\top X\right)^{-1}\right).$$

- We then have $\mathrm{E}[\hat{\beta}] = \beta$ and for any $\hat{\beta}_j$ we have $\mathrm{E}[\hat{\beta}_j] = \beta_j$.
- We also have

$$\mathrm{Var}[\hat{\beta}] = \sigma^2\left(X^\top X\right)^{-1}$$

and for any $\hat{\beta}_j$ we have

$$\mathrm{Var}[\hat{\beta}_j] = \sigma^2 C_{jj}$$

where

$$C = \left(X^\top X\right)^{-1}$$

Sampling distribution for $\hat{\beta}$

and the elements of C are denoted

$$C = \begin{bmatrix} C_{00} & C_{01} & C_{02} & \cdots & C_{0(p-1)} \\ C_{10} & C_{11} & C_{12} & \cdots & C_{1(p-1)} \\ C_{20} & C_{21} & C_{22} & \cdots & C_{2(p-1)} \\ \vdots & \vdots & \vdots & & \vdots \\ C_{(p-1)0} & C_{(p-1)1} & C_{(p-1)2} & \cdots & C_{(p-1)(p-1)} \end{bmatrix}.$$

- The standard error for the $\hat{\beta}$ vector is given by

$$\mathrm{SE}[\hat{\beta}] = s_e \sqrt{\mathrm{diag}\,(X^\top X)^{-1}}$$

and for a particular $\hat{\beta}_j$

$$\mathrm{SE}[\hat{\beta}_j] = s_e \sqrt{C_{jj}}.$$

Sampling distribution for $\hat{\beta}$

- In R

```
C <- solve(t(X) %*% X)
s<-sqrt(sum((y - y_hat) ^ 2) / (n - p))
std.err<-s*sqrt(diag(C))
std.err
[1] 2.8636189207 0.0005416258 0.0797498764
t(summary(pfit)$coef[,2])
     (Intercept)  body_mass_g bill_length_mm
[1,]    2.863619 0.0005416258     0.07974988
sqrt(diag(vcov(pfit)))
   (Intercept)    body_mass_g bill_length_mm
 2.8636189207   0.0005416258   0.0797498764
```

Sampling distribution for $\hat{\beta}$

- Each of the $\hat{\beta}_j$ follows a Gaussian distribution

$$\hat{\beta}_j \sim \mathcal{N}\left(\beta_j, \sigma^2 C_{jj}\right)$$

and

$$\frac{\hat{\beta}_j - \beta_j}{s_e \sqrt{C_{jj}}} \sim t_{n-p}.$$

Single Parameter Tests

- The first test we will see is a test for a single β_j.

$$\mathrm{H}_0 : \beta_j = 0 \quad \text{vs} \quad \mathrm{H}_A : \beta_j \neq 0$$

- The test statistic (TS) takes the form

$$\mathrm{TS} = \frac{\mathrm{EST} - \mathrm{HYP}}{\mathrm{SE}},$$

i.e.

$$t = \frac{\hat{\beta}_j - \beta_j}{\mathrm{SE}[\hat{\beta}_j]} = \frac{\hat{\beta}_j - 0}{s_e\sqrt{C_{jj}}},$$

which, under the null hypothesis, follows a t distribution with $n - p$ degrees of freedom.

Single Parameter Tests

- Example: for `penguins`

$$Y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \varepsilon_i, \qquad i = 1, 2, \ldots, n$$

  where $\varepsilon_i \sim N(0, \sigma^2)$.
  - x_{i1} as the body mass of the ith penguin.
  - x_{i2} as the model bill length of the ith penguin.
- Then the test

$$H_0 : \beta_1 = 0 \quad \text{vs} \quad H_1 : \beta_1 \neq 0$$

  can be found in the `summary` function, in particular:

Single Parameter Tests

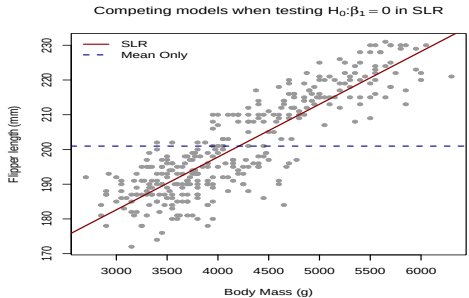
```
summary(pfit)$coef
```

	Estimate	Std. Error	t value	Pr(>|t|)
(Intercept)	122.26867040	2.8636189207	42.697256	1.900760e-136
body_mass_g	0.01301733	0.0005416258	24.033815	1.737931e-74
bill_length_mm	0.54403527	0.0797498764	6.821769	4.306811e-11

- The p-value (last column) given here is specifically for a two-sided test, where the value of the parameter under H_0 is 0.
- Also note in this case, under the hypothesis that $\beta_1 = 0$, the null and alternative essentially specify two different models:
  - H_0: $Y = \beta_0 + \beta_2 x_2 + \varepsilon$
  - H_1: $Y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \varepsilon$
- We are not simply testing whether or not there is a relationship between body mass and flipper length. We are testing if there is a relationship between body mass and flipper length, given that a term for bill length is in the model.

Single Parameter Tests

- Note: when we are in the simple regression case (p=2) testing for $\beta_1 = 0$ corresponds to testing for the need to carrying out a regression in general: if we can not reject $H_0 : \beta_1 = 0$ it means we are better off assuming the variation in Y is better explained by a unique value: the mean. A slightly different intepretation



Confidence Intervals - parameters

- We have already shown that

$$\frac{\hat{\beta}_j - \beta_j}{s_e\sqrt{C_{jj}}} \sim t_{n-p}.$$

- From this we can construct confidence intervals for each of the $\hat{\beta}_j$:

$$\hat{\beta}_j \pm t_{\alpha/2,n-p} \cdot s_e\sqrt{C_{jj}}$$

- In R

```
confint(pfit, level = 0.99)
                      0.5 %       99.5 %
(Intercept)    114.84957959 129.68776122
body_mass_g      0.01161408   0.01442058
bill_length_mm   0.33741855   0.75065200
```

Confidence Intervals - multivariate parameters

- We use the diagonal elements of $\hat{\beta}$ to construct the confidence intervals for β_j
- On the hand we can estimate the covariance between the estimates:

```
vcov(pfit)
```

```
                (Intercept)   body_mass_g bill_length_mm
(Intercept)     8.200313323 -1.140710e-04  -0.1726797535
body_mass_g    -0.000114071  2.933585e-07  -0.0000254611
bill_length_mm -0.172679754 -2.546110e-05   0.0063600428
```

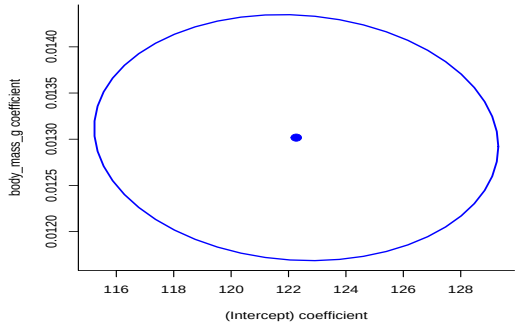
Confidence Intervals - multivariate parameters

- Might be easier to look at the correlation:

```
cov2cor(vcov(pfit))
                (Intercept) body_mass_g bill_length_mm
(Intercept)      1.00000000 -0.07354629     -0.7561295
body_mass_g     -0.07354629  1.00000000     -0.5894511
bill_length_mm -0.75612949 -0.58945111      1.0000000
```

- If we are interested in the confidence interval of **several** parameters we should take into account the correlation between estimates
- For a pair of parameters, say (β_0, β_1) we can construct bivariate confidence intervals which are constructed based on the bivariate normal distribution

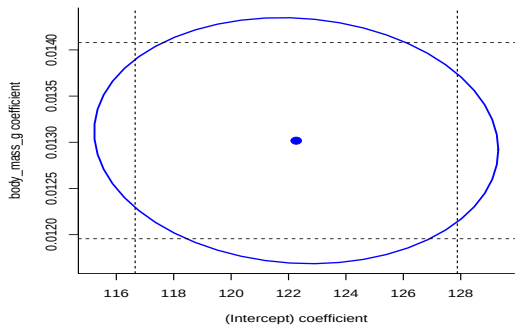
Confidence Intervals - multivariate parameters



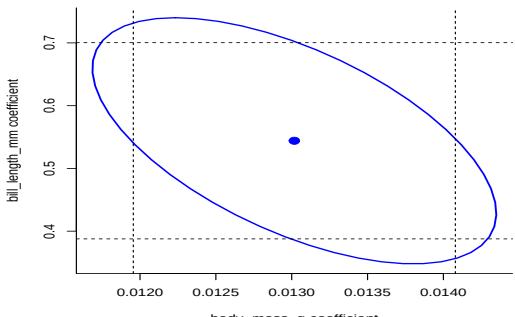
- the ellipse displays a 95% joint confidence region for the regression coefficients β_0 and β_1

Confidence Intervals - multivariate parameters

- the orientation of the ellipse reflects the correlation between the estimates (when estimates are not correlated the ellipse will look like a circle).



Confidence Intervals - multivariate parameters



- individual confidence intervals are often overconfident
- with bi-variate intervals we can notice pairs of (β_0, β_1) whose values in combination are not within the confidence intervals

Confidence Intervals - the regression function

- We can create confidence intervals for the regression function $\mathrm{E}[Y \mid X = x]$.
- We define the vector x_0 to be

$$x_0 = \begin{bmatrix} 1 \\ x_{01} \\ x_{02} \\ \vdots \\ x_{0(p-1)} \end{bmatrix}.$$

- The estimate of $\mathrm{E}[Y \mid X = x_0]$ is given by

$$\begin{aligned} \hat{y}(x_0) &= x_0^\top \hat{\beta} = \\ &= \hat{\beta}_0 + \hat{\beta}_1 x_{01} + \hat{\beta}_2 x_{02} + \cdots + \hat{\beta}_{p-1} x_{0(p-1)}. \end{aligned}$$

Confidence Intervals - the regression function

- This is an unbiased estimate:

$$\mathbb{E}\left[\hat{y}(x_0)\right] = \mathbb{E}\left[x_0^\top \hat{\beta}\right] = x_0^\top \beta = \\ = \beta_0 + \beta_1 x_{01} + \beta_2 x_{02} + \cdots + \beta_{p-1} x_{0(p-1)}$$

- To make an interval estimate, we will also need its standard error:

$$\mathrm{SE}[\hat{y}(x_0)] = \mathrm{SE}[x_0^\top \hat{\beta}] = s_e \sqrt{x_0^\top \left(X^\top X\right)^{-1} x_0}$$

Putting it all together, we obtain (notice we use the t distribution because we are using s_e rather than σ):

The confidence interval for $m(x)$

$$\hat{y}(x_0) \pm t_{\alpha/2, n-p} \cdot s_e \sqrt{x_0^\top \left(X^\top X\right)^{-1} x_0}$$

- In R we use `predict`.

Confidence Intervals - the regression function

- We create a data frame for two additional penguins: both with a body mass of 5000 grams and a bill length of 35 and 45 mm.

```
new.peng <- data.frame(body_mass_g = c(5000, 5000),
    bill_length_mm = c(45, 35)); new.peng
```

```
  body_mass_g bill_length_mm
1        5000            45
2        5000            35
```

- We use the `predict` function with `interval = "confidence"` to obtain intervals for the mean flipper length of these two penguins.

```
(cint <- predict(pfit, newdata = new.peng,
  interval = "confidence", level = 0.99))
```

```
       fit      lwr      upr
1 211.8369 210.4808 213.1930
2 206.3966 203.5755 209.2176
```

Confidence Intervals - the regression function

- R reports the estimate $\hat{y}(x_0)$ (`fit`) for each, as well as the lower (`lwr`) and upper (`upr`) bounds for the interval at a desired level (99%).
- One of these estimates is good while one is suspect (i.e. very wide)

```
cint[,3] - cint[,2]
```

```
       1        2
2.712218 5.642139
```

- The new observations are within the range of observed values

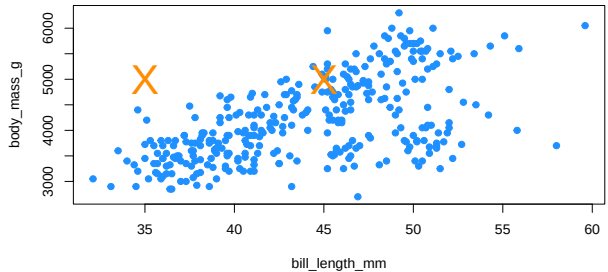
```
range(penguins$body_mass_g)
```

```
[1] 2700 6300
```

```
range(penguins$bill_length_mm)
```

```
[1] 32.1 59.6
```

Confidence Intervals - the regression function



- However, we have to consider mass and bill length **together**. Based on the above plot, one of the new penguins is within the "blob" of observed values, while the other, is noticeably outside of the observed values. This is a hidden extrapolation which you should be aware of when using multiple regression.

Confidence Intervals - the regression function

- A quick verification of some of the mathematics in R.

```
x0 <- c(1,5000,45)
x0 %*% beta.hat
         [,1]
[1,] 211.8369
x0[1]*beta.hat[1] + x0[2]*beta.hat[2]+ x0[3]*beta.hat[3]
[1] 211.8369
```

Confidence Intervals - the regression function

$$x_0 = \begin{bmatrix} 1 \\ 5000 \\ 45 \end{bmatrix}, \quad \hat{\beta} = \begin{bmatrix} 122.27 \\ 0.013017 \\ 0.54404 \end{bmatrix}$$

$$\hat{y}(x_0) = x_0^\top \hat{\beta} = \begin{bmatrix} 1 & 5000 & 45 \end{bmatrix} \begin{bmatrix} 122.27 \\ 0.013017 \\ 0.54404 \end{bmatrix} = 211.8369$$

Confidence Intervals - the regression function

- Note that, using a particular value for x_0, we can essentially extract certain $\hat{\beta}_j$ values.

```
x0 <- c(0, 0, 1)
x0 %*% beta.hat
          [,1]
[1,] 0.5440353
```

- With this in mind, confidence intervals for the individual $\hat{\beta}_j$ are actually a special case of a confidence interval for mean response.
- Notice that the confidence interval gives information on the $E[Y|X = x_0]$: this is the typical value we can expect at some predictor value x_0
- What about the actual values of Y? $\rightarrow$ We have $(Y|X = x_o) = X_0\beta + \varepsilon$: prediction intervals

Prediction Intervals

- We use $\hat{y}(x_0)$ to predict Y_0, a new observation of Y at the predictor vector x_0.

$$\begin{aligned}\hat{y}(x_0) &= x_0^\top \hat{\beta} \\ &= \hat{\beta}_0 + \hat{\beta}_1 x_{01} + \hat{\beta}_2 x_{02} + \cdots + \hat{\beta}_{p-1} x_{0(p-1)}\end{aligned}$$

$$\begin{aligned}\mathrm{E}[\hat{y}(x_0)] &= x_0^\top \beta \\ &= \beta_0 + \beta_1 x_{01} + \beta_2 x_{02} + \cdots + \beta_{p-1} x_{0(p-1)}\end{aligned}$$

- We need to account for the additional variability of an observation about its mean:

$$\mathrm{SE}[\hat{y}(x_0) + \varepsilon] = s_e \sqrt{1 + x_0^\top \left(X^\top X\right)^{-1} x_0}$$

Prediction Intervals

- Then we arrive at

The prediction interval for $m(x_0)$

$$\hat{y}(x_0) \pm t_{\alpha/2,n-p} \cdot s_e \sqrt{1 + x_0^\top \left(X^\top X\right)^{-1} x_0}$$

- in R

```
predict(pfit, newdata = new.peng,
        interval = "prediction", level = 0.99)
```

```
       fit      lwr      upr
1 211.8369 195.1506 228.5233
2 206.3966 189.5279 223.2653
```

Confidence and Prediction Intervals in SLR

In the SLR case the design matrix X is

$$\begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix} \quad \text{and} \quad \left(X^\top X\right) = \begin{bmatrix} n & \sum_{i=1}^n x_i \\ \sum_{i=1}^n x_i & \sum_{i=1}^n x_i^2 \end{bmatrix}$$

From this one can derive:

$$\left(X^\top X\right)^{-1} = \frac{1}{n^2 s_x^2} \begin{bmatrix} \sum x_i^2 & -\sum x_i \\ -\sum x_i & n \end{bmatrix}$$

And obtain the same value for the SD(β_0) and SD(β_1) seen for SLR.

Confidence and Prediction Intervals in SLR

Furthermore, for the SLR we can derive that the variability of $\hat{m}(x_0)$ as

$$\text{SD}^2(\hat{m}(x_0)) = s_e^2 * x_0^\top \left(X^\top X\right)^{-1} x_0 = s_e^2 \left(\frac{1}{n} + \frac{(x-\bar{x})^2}{\sum_{i=1}^n (x_i - \bar{x})^2}\right)$$

From this we can derive

The confidence interval for $m(x)$ in SLR

$$\hat{m}(x_0) \pm t_{\alpha/2,n-2} \times s_e \sqrt{\frac{1}{n} + \frac{(x_0-\bar{x})^2}{\sum_{i=1}^n (x_i - \bar{x})^2}}$$

and

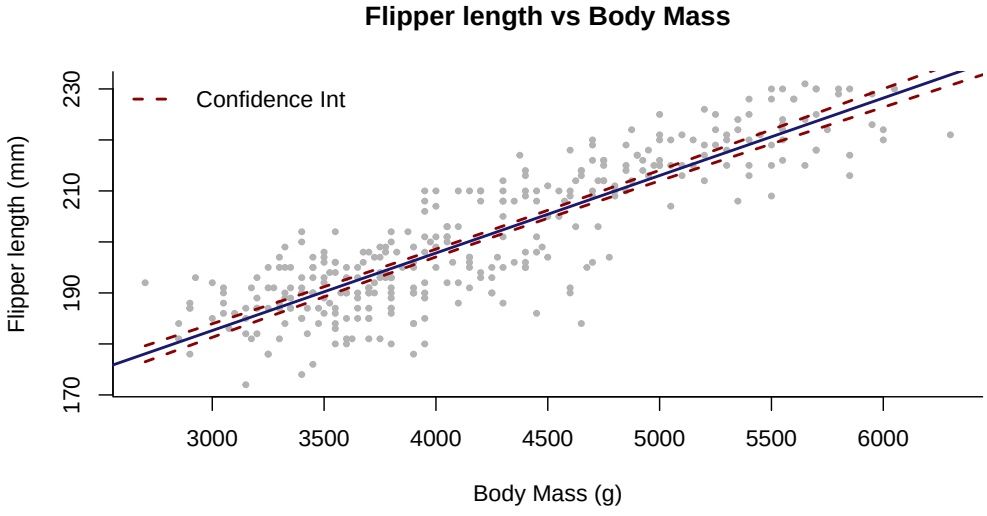
Confidence and Prediction Intervals in SLR

The prediction interval for $m(x)$ in SLR

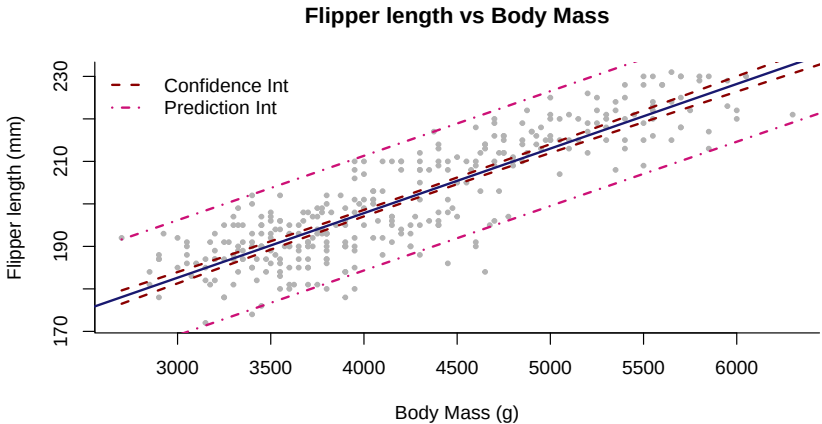
$$\hat{m}(x_0) \pm t_{\alpha/2,n-2} \times s_e \sqrt{1 + \frac{1}{n} + \frac{(x_0 - \bar{x})^2}{\sum_{i=1}^{n}(x_i - \bar{x})^2}}.$$

Notice how these depend on s_e, $(x_0 - \bar{x})$ and n.
Take the SLR case of predicting flipper length from body mass only.

Confidence and Prediction Intervals in SLR



Confidence and Prediction Intervals in SLR



Model selection [2]

[2] Material in these slides was heavily influenced by David Dalpiaz *Applied Statistics with R!*, the book is under active development.

Some important questions

- Is at least one of the predictors X_1, X_2, ..., X_{p-1} useful in predicting the response?
- Do all the predictors help to explain Y, or is only a subset of the predictors useful?
- How well does the model fit the data?

- We can still use the decomposition of variation that we had seen in the simple linear regression:

$$\sum_{i=1}^{n}(y_i - \bar{y})^2 = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \sum_{i=1}^{n}(\hat{y}_i - \bar{y})^2$$

  That is:

$$\mathrm{SS}_{tot} = \mathrm{SS}_{res} + \mathrm{SS}_{reg}.$$

Significance of regression

- This means that, we can still calculate R^2 in the same manner as before, which R continues to do automatically.

```
pfit$call
```

```
lm(formula = flipper_length_mm ~ body_mass_g + bill_length_mm,
    data = penguins)
```

```
summary(pfit)$r.squared
```

```
[1] 0.7914954
```

- The interpretation changes slightly compared to the simple linear regression. In the multiple linear regression case, we say that 79.15 of the observed variation in the penguin's flipper's length is explained by the linear relationship with the two predictor variables, mass and bill length.

Significance of regression

- But R^2 is not something that can give a clear indication that the regression captures a considerable proportion of the variability - there is no obvious cut-off to know when the value is "large"
- We wish to have a way to declare that the model is useful and have a test to say whether the model is significantly different from taking the much simpler model in which we include no predictor variable
- In multiple regression, the **significance** of regression is tested using

$$\mathrm{H}_0 : \beta_1 = \beta_2 = \cdots = \beta_{p-1} = 0.$$

- The "model under the null hypothesis" is

$$Y_i = \beta_0 + \epsilon_i.$$

i.e. none of the predictors have a significant linear relationship with the response.

Significance of regression

- We will denote the predicted values of this model as $\hat{y}_{0,i}$ and we have

$$\hat{y}_{0,i} = \bar{y}.$$

- The alternative hypothesis here is that

$$\mathrm{H}_A : \text{At least one of } \beta_j \neq 0, j = 1, 2, \cdots, (p-1)$$

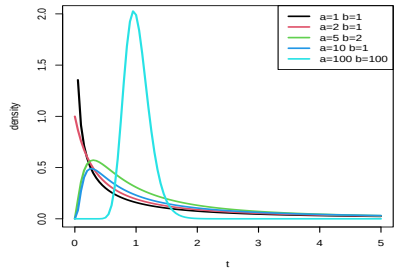
- The "model under the alternative hypothesis" is

$$Y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_{(p-1)} x_{i(p-1)} + \epsilon_i$$

  i.e. there is some linear relationship between y and the predictors, $x_1, x_2, \ldots, x_{p-1}$.
- We will denote the predicted values of this model as $\hat{y}_{A,i}$.

F distribution

- $f(t) = \frac{\sqrt{\frac{(at)^a \, b^b}{(at+b)^{a+b}}}}{t \, \mathrm{B}\left(\frac{a}{2}, \frac{b}{2}\right)}$
- a and b **degrees of freedom**
- $\mathrm{B}(x, y) = \int_0^1 t^{x-1}(1-t)^{y-1}\, dt$ is the **beta function**
- In R `df`, `pf`, `qf` and `rf` (`*f`)



The F-distribution arises as the ratio of two independent scaled chi-square distributions. So if $X_1 \sim \chi^2_{d1}$ and $X_2 \sim \chi^2_{d2}$:

$$\frac{X_1/d_1}{X_2/d_2} \sim F_{d1,d2}$$

The domain is the positive reals.

ANOVA table

- To develop the test for the significance of the regression, we will arrange the variance decomposition into an **ANalysis Of VAriance** (**ANOVA**) table:

Source	Sum of Squares	Degrees of Freedom	Mean Square	F
Reg.	$\sum_{i=1}^{n}(\hat{y}_{A,i}-\bar{y})^2$	$p-1$	$\mathrm{SS}_{reg}/(p-1)$	$\mathrm{MS}_{reg}/\mathrm{MS}_{res}$
Error	$\sum_{i=1}^{n}(y_i-\hat{y}_{A,i})^2$	$n-p$	$\mathrm{SS}_{res}/(n-p)$	
Total	$\sum_{i=1}^{n}(y_i-\bar{y})^2$	$n-1$		

- We calculate the F-statistic

$$F=\frac{\sum_{i=1}^{n}(\hat{Y}_{A,i}-\bar{Y})^2/(p-1)}{\sum_{i=1}^{n}(Y_i-\hat{Y}_{A,i})^2/(n-p)},$$

under H_0 the distribution of the statistics is a **F-distribution**, also known as **Snedecor's F distribution** or the **Fisher-Snedecor distribution** with degrees of freedom $p-1$ and $n-p$ (Notation $F\sim F_{p-1,n-p}$)

ANOVA table

- A large value of the statistic corresponds to a large portion of the variance being explained by the regression. We reject H_0 for large values of F and the p-value is calculated as

$$\Pr(F > F_{obs})$$

- In `R`, we first explicitly specify the two models in `R` and save the results in different variables.
- We then use `anova` to compare the two models
  - Model under H_0: $Y_i = \beta_0 + \epsilon_i$
  - Model under H_A: $Y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \epsilon_i$

ANOVA table

- In R

```
null_pengs = lm(flipper_length_mm ~ 1,data = penguins)
full_pengs = lm(flipper_length_mm ~ body_mass_g + bill_length_mm,data = penguins
anova(null_pengs, full_pengs)
```

```
Analysis of Variance Table

Model 1: flipper_length_mm ~ 1
Model 2: flipper_length_mm ~ body_mass_g + bill_length_mm
  Res.Df   RSS Df Sum of Sq      F    Pr(>F)
1    332 65219
2    330 13598  2     51620 626.35 < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

ANOVA table

- We see that the value of the F statistic is 626.35, and the p-value is extremely low, so we reject the null hypothesis at any reasonable α and say that the regression is significant. At least one of `body_mass_g` or `bill_length_mm` has a useful linear relationship with `flipper_length_mm`.

ANOVA table

- Now consider the summary

```
pfit <- lm(flipper_length_mm ~ body_mass_g + bill_length_mm,data = penguins)
summary(pfit)
```

```
Call:
lm(formula = flipper_length_mm ~ body_mass_g + bill_length_mm,
    data = penguins)

Residuals:
     Min       1Q   Median       3Q      Max
-20.9869  -4.5868   0.4285   4.8784  14.0977

Coefficients:
                Estimate  Std. Error t value        Pr(>|t|)
(Intercept)    122.2686704   2.8636189  42.697         < 2e-16 ***
body_mass_g      0.0130173   0.0005416  24.034         < 2e-16 ***
bill_length_mm   0.5440353   0.0797499   6.822 0.0000000000431 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

ANOVA table

```
Residual standard error: 6.419 on 330 degrees of freedom
Multiple R-squared:  0.7915,        Adjusted R-squared:  0.7902
F-statistic: 626.3 on 2 and 330 DF,  p-value: < 2.2e-16
```

- Notice that the value reported in the row for `F-statistic` is indeed the F test statistic for the significance of regression test, and additionally it reports the two relevant degrees of freedom.
- Verify the sums of squares and degrees of freedom directly in `R`.

```
sum((fitted(full_pengs) - fitted(null_pengs)) ^ 2) # SSreg
```

```
[1] 51620.25
```

```
sum(resid(full_pengs) ^ 2)# SSres
```

```
[1] 13598.38
```

```
sum(resid(null_pengs) ^ 2)# SStot
```

```
[1] 65218.64
```

```
#
# Degrees of Freedom: Regression
length(coef(full_pengs)) - length(coef(null_pengs))
```

ANOVA table

```
[1] 2
```

```
 # Degrees of Freedom: Error
 length(resid(full_pengs)) - length(coef(full_pengs))
```

```
[1] 330
```

```
 # Degrees of Freedom: Total
 length(resid(null_pengs)) - length(coef(null_pengs))
```

```
[1] 332
```

Nested Models

- The significance of regression test is actually a special case of testing what we will call **nested models**
- One model is "nested" inside the other means that one model contains a subset of the predictors from only the larger model.
- Consider the following full model,

$$Y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \cdots + \beta_{(q-1)} x_{i(q-1)} + \cdots + \beta_{(p-1)} x_{i(p-1)} + \epsilon_i$$

This model has $p-1$ predictors, so p β-parameters. We will denote the predicted values of this model as $\hat{y}_{A,i}$.

Nested Models

- Let the null model be

$$Y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_{(q-1)} x_{i(q-1)} + \epsilon_i$$

  where $q < p$. This model has $q - 1$ predictors, so q β-parameters. We will denote the predicted values of this model as $\hat{y}_{0i}$.

- The difference between models can be codified by the null hypothesis

$$\mathrm{H}_0 : \beta_q = \beta_{q+1} = \cdots = \beta_{p-1} = 0.$$

  which is contrasted to

$$\mathrm{H}_A : \text{At least one } \beta_j \neq 0,\ j = q, \ldots, p-1$$

Nested Models

- Denote $\text{SS}_{res}(\text{H}_0)$, the sum of squared of residuals under H_0 and $\text{SS}_{res}(\text{H}_A)$ the sum of squared of residuals under H_A
- The fit of the smaller model in general is such that

$$\text{SS}_{res}(\text{H}_0) \geq \text{SS}_{res}(\text{H}_A)$$

- If $\text{SS}_{res}(\text{H}_0) - \text{SS}_{res}(\text{H}_A)$ is small, then the fit of the smaller model is almost as good as the larger model and so we would prefer the smaller (more parsimonoius) model on the grounds of simplicity.
- On the other hand, if the difference is large, then the superior fit of the larger model would be preferred. This suggests that something like:

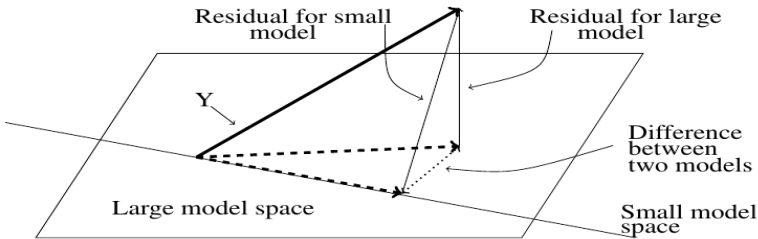
$$\frac{\text{SS}_{res}(\text{H}_0) - \text{SS}_{res}(\text{H}_A)}{\text{SS}_{res}(\text{H}_A)}$$

would be a potentially good test statistic where the denominator is used for scaling purposes.

Nested Models

- It turns out that

$$\begin{aligned} \mathrm{SS}_{res}(\mathrm{H}_0) - \mathrm{SS}_{res}(\mathrm{H}_A) &= \sum_{i=1}^{n}(y_i - \hat{y}_{o,i})^2 - \sum_{i=1}^{n}(y_i - \hat{y}_{A,i})^2 \\ &= \sum_{i=1}^{n}(\hat{y}_{A,i} - \hat{y}_{o,i})^2 \end{aligned}$$



Nested Models

- We can then perform this test using an F-test, typically shown in an **ANOVA table**.

Source	Sum of Squares	Degrees of Freedom	Mean Square	F
diff	$\sum_{i=1}^{n}(\hat{y}_{A,i} - \hat{y}_{0,i})^2$	$p - q$	$\text{SS}_{diff}/(p-q)$	$\text{MS}_{diff}/\text{MS}_{res}$
full	$\sum_{i=1}^{n}(y_i - \hat{y}_{A,i})^2$	$n - p$	$\text{SS}_{res}/(n-p)$	
null	$\sum_{i=1}^{n}(y_i - \hat{y}_{0,i})^2$	$n - q$		

$$
\begin{aligned}
F &= \frac{(\text{SS}_{res}(\text{H}_0) - \text{SS}_{res}(\text{H}_A))/(p-q)}{\text{SS}_{res}(\text{H}_A)/(n-p)} \\
&= \frac{\sum_{i=1}^{n}(\hat{y}_{A,i} - \hat{y}_{0,i})^2/(p-q)}{\sum_{i=1}^{n}(y_i - \hat{y}_{A,i})^2/(n-p)}.
\end{aligned}
$$

- Notice that the row for *diff* compares the sum of the squared differences of the predicted values. The degrees of freedom is the difference in the number of β-parameters estimated in the two models.

Nested Models

- For example, the penguins dataset has a number of additional variables that we have yet to use.

```
names(penguins)
```

```
[1] "species"           "island"            "bill_length_mm"
[4] "bill_depth_mm"     "flipper_length_mm" "body_mass_g"
[7] "sex"               "year"
```

- We will consider two different models:
  - Full: `flipper_length_mm ~bill_length_mm+body_mass_g+bill_depth_mm+year`
  - Null: `flipper_length_mm ~bill_length_mm+body_mass_g`

i.e.

$$\mathrm{H}_0 : \beta_{\texttt{bd}} = \beta_{\texttt{year}} = 0$$

The alternative is that at least one of the β_j from the null is not 0.

Nested Models

- In R

```
null_pengs <- lm(flipper_length_mm ~ body_mass_g + bill_length_mm, data =
full_pengs <- lm(flipper_length_mm ~
      body_mass_g + bill_length_mm+bill_depth_mm+year, data = penguins)
anova(null_pengs, full_pengs)
```

```
Analysis of Variance Table

Model 1: flipper_length_mm ~ body_mass_g + bill_length_mm
Model 2: flipper_length_mm ~ body_mass_g + bill_length_mm + bill_depth_mm
    year
  Res.Df   RSS Df Sum of Sq      F    Pr(>F)
1    330 13598
2    328 10055  2    3543.3 57.792 < 2.2e-16 ***
```

Nested Models

```
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

- The F statistic is 57.792, and the p-value is pretty small, so we can reject the null hypothesis at any reasonable α and say that one of `bill_depth_mm` or `year` are significant when `bill_length_mm` and `body_mass_g` are already in the model.

Nested Models

- Verification in R

```
 (ssdiff <- sum((fitted(full_pengs) - fitted(null_pengs)) ^ 2)) # SSdiff
[1] 3543.322
 sum(resid(full_pengs) ^ 2) # SSR (For Full)
[1] 10055.06
 sum(resid(null_pengs) ^ 2) # SSR (For Null)
[1] 13598.38
 # Degrees of Freedom: Diff
 (dfdiff <- length(coef(full_pengs)) - length(coef(null_pengs)))
[1] 2
 # Degrees of Freedom: Full
 length(resid(full_pengs)) - length(coef(full_pengs))
[1] 328
 # Degrees of Freedom: Null
 length(resid(null_pengs)) - length(coef(null_pengs))
[1] 330
 # F value
 (ssdiff/dfdiff) / (sum(resid(full_pengs) ^ 2)/full_pengs$df.resid)
[1] 57.79227
```

Quality Criterion

- We wish to find a model that explain large parts of the variance of the original data: measure this with R^2:

```
summary(full_pengs)$ r.squared
[1] 0.8458253
summary(null_pengs)$ r.squared
[1] 0.7914954
```

- The added variables are not significant, why does R^2 increase?
- Overfitting: increasing the number of predictors always increases R^2:

```
(n <- nrow(penguins))
[1] 333
useless_covariates <- matrix(rnorm(n*(n-1)), ncol = n-1)
summary(lm(penguins$flipper_length_mm ~ useless_covariates))$r.squared
[1] 1
```

Quality Criterion

- A perfect fit! It does have some issues though

```
length(coef(lm(penguins$flipper_length_mm ~ useless_covariates)))
[1] 333
summary(lm(penguins$flipper_length_mm ~ useless_covariates))$coef[1:3,]
                    Estimate Std. Error t value Pr(>|t|)
(Intercept)         188.76824        NaN     NaN      NaN
useless_covariates1  18.61910        NaN     NaN      NaN
useless_covariates2  26.92296        NaN     NaN      NaN
summary(lm(penguins$flipper_length_mm ~ useless_covariates))$sigma
[1] NaN
```

- Criteria for assessing quality of fit such as R^2 and RMSE have a fatal flaw: it is impossible to add a predictor to a model and make R^2 or RMSE worse.

Quality Criterion

- This suggests that we need a quality criteria that takes into account the size of the model, since our preference is for small models that still fit well.
- Sacrifice a small amount of "goodness-of-fit" to obtain a smaller model.

Quality Criterion

- We will look at three criteria that do this explicitly: AIC, BIC, and Adjusted R^2.
- We will also look at one, Cross-Validated RMSE, which implicitly considers the size of the model.
- We will look at frameworks that also prevents overfitting

Information Criteria

- An information criterion balances the fitness of a model with the number of predictors employed.

$$\text{IC} \quad = \quad \text{goodness of fit} + \text{penalty for the complexity}$$

- Complexity of a model is a function of number of parameter $p(\mathcal{M})$
- The goodness of fit can be taken to be the negative log-likelihood: we have chosen the model which minimize its value.
- We use ICs of the form

$$\text{IC} \quad = \quad \text{-2*logLik}(\mathcal{M}) + k * p(\mathcal{M})$$

- Choose models which minimize IC: balance good fit with complexity

Information Criteria

- For a linear regression model, the maximized likelihood reduces to a function of the sum of square of residuals $\mathrm{SS}_{res}(\mathcal{M})$ after fitting the model $\mathcal{M}$ since:

$$\begin{aligned} lik(\hat{\beta}, \hat{\sigma}; y, x) &= \sum_{i=1}^{n} \left\{ -\log 2\pi - \log(\hat{\sigma}) - \frac{1}{2} \left(\frac{y - \hat{y}_i}{\hat{\sigma}} \right)^2 \right\} = \\ &= -n \log 2\pi - \frac{n}{2} \log \frac{\sum_{i=1}^{n} (y - \hat{y}_i)^2}{n} - \frac{1}{2} n = \\ &= constant - \frac{n}{2} \log \mathrm{MSS}_{res}(\mathcal{M}) \end{aligned}$$

  because the maximum likelihood estimator of σ is $\hat{\sigma} = (\sum_{i=1}^{n} (y - \hat{y}_i)^2)/n$

- (In rare cases ICs are written as $\mathrm{IC} = 2^* \mathrm{logLik}(\mathcal{M}) - k * p(\mathcal{M})$: make sure you know how the software you are using defines them)

Information Criteria

- For linear models therefore we have:

Akaike's Information Criterion (AIC)

$$AIC(\mathcal{M}) \quad = \quad n \times \log \mathrm{MSS}_{res}(\mathcal{M}) + 2p(\mathcal{M})$$

Bayesian Information Criterion (BIC)

$$BIC(\mathcal{M}) \quad = \quad n \times \log \mathrm{MSS}_{res}(\mathcal{M}) + \log(n)p(\mathcal{M})$$

- The BIC replaces the 2 in the AIC penalization with $\log(n)$ so it penalizes more complex models more (since $log(n) > 2$ if $n \geq 8$).

Information Criteria

- This is one of the reasons why BIC is preferred by some practitioners for model comparison. Also, because it is consistent in selecting the true model: if enough data is provided, the BIC is guaranteed to select the data-generating model among a list of candidate models.
- The AIC and BIC can be computed in `R` through the functions `AIC` and `BIC`. They take a model as the input.

```
# Two models with different predictors
mod1 <- lm(flipper_length_mm ~ body_mass_g + bill_length_mm, data = pengu
mod2 <- lm(flipper_length_mm ~ body_mass_g + bill_length_mm +
      bill_depth_mm, data = penguins)
# AICs
c(AIC(mod1) , AIC(mod2) )
```

```
[1] 2188.298 2119.035
```

Information Criteria

```
# BICs
c(BIC(mod1) , BIC(mod2) )
```

```
[1] 2203.530 2138.075
```

```
## consider adding bill depth
# BIC can also be derived as - see ?AIC
AIC(mod1, k=log(nrow(penguins)))
```

```
[1] 2203.53
```

```
summary(mod2)
```

Information Criteria

```
Call:
lm(formula = flipper_length_mm ~ body_mass_g + bill_length_mm +
    bill_depth_mm, data = penguins)

Residuals:
     Min       1Q   Median       3Q      Max
-21.6181  -3.7380   0.1721   4.0493  14.4735

Coefficients:
                Estimate  Std. Error t value Pr(>|t|)
(Intercept)    156.7821741   4.6704731  33.569  < 2e-16 ***
body_mass_g      0.0109672   0.0005395  20.327  < 2e-16 ***
bill_length_mm   0.5884351   0.0719407   8.179 6.28e-15 ***
bill_depth_mm   -1.6220179   0.1830620  -8.860  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 5.777 on 329 degrees of freedom
Multiple R-squared:  0.8317,        Adjusted R-squared:  0.8301
F-statistic: 541.8 on 3 and 329 DF,  p-value: < 2.2e-16
```

Adjusted R^2

- For a least squares model with p variables, the adjusted R^2 statistic is calculated as

$$\text{Adjusted } R^2 = 1 - \frac{\text{SS}_{res}/(n-p-1)}{\text{SS}_{tot}/(n-1)}$$

- ICs: small value indicates a low test error. Adjusted R^2: high value indicates a model with a small test error.
- Maximizing the adjusted R^2 is equivalent to minimizing $\text{SS}_{res}/(n-p-1)$.
- While SS_{res} always decreases as the number of the variables in the model increases, $\text{SS}_{res}/(n-p-1)$ may increase or decrease, due to the presence of p in the denominator.
- Unlike the R^2 statistic, the adjusted R^2 statistic pays a price for the inclusion of unnecessary variables in the model
- The adjusted R^2 can be computed in `R` through the functions `summary`

Adjusted R^2

```
summary(full_pengs)$adj.r.squared
```

```
[1] 0.8439452
```

```
summary(null_pengs)$adj.r.squared
```

```
[1] 0.7902318
```

- The improvement in the goodness of fit is large enough to justify the increase in model complexity.

Variable selection

- Find the simplest model to explain the data. Smaller models lead to less variable inference/prediction and can be more explainable than complex models. Use Occam's Razor principle: the smallest model that fits the data is best.
- Simplest model for best fit: use any of the criteria discussed before to choose.
- The most direct approach is called all subsets or best subsets regression: compute the least squares fit for all possible subsets and then choose between them based on some criterion.
- However we often can not examine all possible models, since they are 2^{p-1} of them; for example when $p-1=30$ there are 1073741824 models!
- Instead we need an automated approach that searches through a subset of them. We discuss some commonly used approaches next.

Forward stepwise selection

1. Let $\mathcal{M}_0$ denote the null model, which contains no predictors.
2. For $k = 1, \ldots, p-2$:
   - Consider all $p-k$ models that augment the predictors in $\mathcal{M}_k$ with one additional predictor.
   - Choose the best among these $p-k$ models, and call it $\mathcal{M}_{k+1}$. Here best is defined as having smallest SS_{res} .
3. Select a single best model from among $\mathcal{M}_0, \ldots, \mathcal{M}_{p-1}$ using cross-validated prediction error, AIC, BIC, or adjusted R^2 .

- Computational advantage over best subset selection is clear.
- It is not guaranteed to find the best possible model out of all 2^{p-1} models containing subsets of the $p-1$ predictors.

Backward stepwise selection

1. Let $\mathcal{M}_{p-1}$ denote the full model, which contains all $p-1$ predictors.
2. For $k = p-1, p-2, \ldots, 1$:
   - Consider all k models that contain all but one of the predictors in $\mathcal{M}_k$, for a total of $k-1$ predictors
   - Choose the best among these k models, and call it $\mathcal{M}_{k-1}$. Here best is defined as having smallest SS_{res} .
3. Select a single best model from among $\mathcal{M}_0, \ldots, \mathcal{M}_{p-1}$ using cross-validated prediction error, AIC, BIC, or adjusted R^2 .

Backward stepwise selection

- Like forward stepwise selection, the backward selection approach searches through only $1 + p(p-1)/2$ models, and so can be applied in settings where p is too large to apply best subset selection
- Like forward stepwise selection, backward stepwise selection is not guaranteed to yield the best model containing a subset of the $p-1$ predictors.
- Backward selection requires that the number of samples n is larger than the number of variables $p-1$ (so that the full model can be fit). In contrast, forward stepwise can be used even when $n < p-1$, and so is the only viable subset method when p is very large.

Forward and Backward selection in R

- Forward search (`trace=0` gives only the final model)

```
null<-lm(flipper_length_mm ~1, data = penguins)
full<-lm(flipper_length_mm ~body_mass_g + bill_length_mm +
      bill_depth_mm + year, data = penguins)
step(null, scope=list(lower=null, upper=full), direction="forward",
   k=2, trace=1)
```

```
Start:  AIC=1759.36
flipper_length_mm ~ 1

                 Df Sum of Sq   RSS    AIC
+ body_mass_g     1     49703 15516 1283.2
+ bill_length_mm  1     27818 37401 1576.2
+ bill_depth_mm   1     21773 43446 1626.1
+ year            1      1488 63730 1753.7
<none>                       65219 1759.4

Step:  AIC=1283.21
flipper_length_mm ~ body_mass_g
```

Forward and Backward selection in R

```
                  Df Sum of Sq   RSS    AIC
+ bill_depth_mm    1    2304.9 13211 1231.7
+ bill_length_mm   1    1917.6 13598 1241.3
+ year             1    1136.6 14379 1259.9
<none>                         15516 1283.2

Step:  AIC=1231.66
flipper_length_mm ~ body_mass_g + bill_depth_mm

                  Df Sum of Sq   RSS    AIC
+ bill_length_mm   1    2232.5 10979 1172.0
+ year             1    1003.7 12208 1207.3
<none>                         13211 1231.7

Step:  AIC=1172.02
flipper_length_mm ~ body_mass_g + bill_depth_mm + bill_length_mm

       Df Sum of Sq   RSS    AIC
+ year  1    923.54 10055 1144.8
```

Forward and Backward selection in R

```
<none>              10979 1172.0

Step:  AIC=1144.76
flipper_length_mm ~ body_mass_g + bill_depth_mm + bill_length_mm +
    year


Call:
lm(formula = flipper_length_mm ~ body_mass_g + bill_depth_mm +
    bill_length_mm + year, data = penguins)

Coefficients:
   (Intercept)     body_mass_g   bill_depth_mm  bill_length_mm            year
   -3969.79600         0.01101        -1.57884         0.57800         2.05479
```

Forward and Backward selection in R

- Backward search

```
step(full, scope=list(lower=null, upper=full),
     direction="backward", k=2, trace=1)
```

```
Start:  AIC=1144.76
flipper_length_mm ~ body_mass_g + bill_length_mm + bill_depth_mm +
    year

                 Df Sum of Sq   RSS    AIC
<none>                       10055 1144.8
- year            1     923.5 10979 1172.0
- bill_length_mm  1    2152.4 12208 1207.3
- bill_depth_mm   1    2477.2 12532 1216.1
- body_mass_g     1   13900.9 23956 1431.8

Call:
lm(formula = flipper_length_mm ~ body_mass_g + bill_length_mm +
    bill_depth_mm + year, data = penguins)

Coefficients:
```

Forward and Backward selection in R

(Intercept)	body_mass_g	bill_length_mm	bill_depth_mm	year
-3969.79600	0.01101	0.57800	-1.57884	2.05479

Stepwise Search

- Stepwise search checks going both backwards and forwards at every step. It considers the addition of any variable not currently in the model, as well as the removal of any variable currently in the model.
- We perform stepwise search using AIC as our criterion.

```
intermediate <- lm(flipper_length_mm ~body_mass_g + bill_length_mm, data = penguins)
step(intermediate, scope = list(lower = null, upper=full),
      direction="both",trace=1,k=2)
```

```
Start:  AIC=1241.28
flipper_length_mm ~ body_mass_g + bill_length_mm

                 Df Sum of Sq   RSS    AIC
+ bill_depth_mm   1    2619.8 10979 1172.0
+ year            1    1066.1 12532 1216.1
<none>                        13598 1241.3
- bill_length_mm  1    1917.6 15516 1283.2
- body_mass_g     1   23802.3 37401 1576.2
```

Stepwise Search

```
Step:  AIC=1172.02
flipper_length_mm ~ body_mass_g + bill_length_mm + bill_depth_mm

                 Df Sum of Sq   RSS    AIC
+ year            1     923.5 10055 1144.8
<none>                        10979 1172.0
- bill_length_mm  1    2232.5 13211 1231.7
- bill_depth_mm   1    2619.8 13598 1241.3
- body_mass_g     1   13788.0 24767 1440.9

Step:  AIC=1144.76
flipper_length_mm ~ body_mass_g + bill_length_mm + bill_depth_mm +
    year

                 Df Sum of Sq   RSS    AIC
<none>                        10055 1144.8
- year            1     923.5 10979 1172.0
- bill_length_mm  1    2152.4 12208 1207.3
- bill_depth_mm   1    2477.2 12532 1216.1
```

Stepwise Search

```
- body_mass_g     1   13900.9 23956 1431.8

Call:
lm(formula = flipper_length_mm ~ body_mass_g + bill_length_mm +
    bill_depth_mm + year, data = penguins)

Coefficients:
   (Intercept)     body_mass_g  bill_length_mm   bill_depth_mm            year
   -3969.79600         0.01101         0.57800        -1.57884         2.05479
```

Variable selection - some perspective

- Automatic methods are tempting but might not yield the "best" model.
- Models still need to be assessed - for example checking the linearity and other assumptions (see also later).
- Stepwise variable selection tends to pick models that are smaller than desirable for prediction purposes.
- One variable being out of the model doesn't mean it is not related to Y: it means other variables relate more or explain the same pattern (for the dataset under study).
- We select a model that *predicts well*. This does not mean the model uncovers a *true relationship* between real-world variables.
- We use the data twice: to select the model and then to do inference - that's cheating! We would be overconfident in our results. The *Inference after Selection* problem.

Inference after Selection

- All statistical theory for the regression models is derived assuming the model is fixed and known **before** seeing the data.
- If we use the data to select a model we will be selecting the "best" variables: significance will be, for example, overstated.
- The data is random, and model selection depends on the data, so it becomes random: the statistical formulas we use do not account for this additional randomness.
- (this is a really complicated issue, even plotting the data to "have a look" is using the data twice)
- How can we solve this?
  - If you are only interested in the "best model" you can disregard this issue, but typically we are interested in inference
  - Use more advanced statistical theory (we don't see that here)
  - Data splitting (assuming you have independent observations): choose the model using a data subset and estimate its final form using another the remaining data points.

Validation-based selection

- Each of the previous three metrics explicitly used p, the number of parameters, in their calculations. Thus, they all explicitly limit the size of models chosen when used to compare models.
- The calculations rely on an assumption of "true" underlying model
- Often times models are needed for out-of-sample prediction: we need to quantify the possible error.
- First up: **cross-validation**.

Validation-based selection

- Fit any model to the dataset available *without* one observation i.
- Predict y_i for the model in which (x_i, y_i) was not included in the model. Denote this with $\hat{y}_{[i]}$.
- Now $e_{[i]}$ is the residual for the ith observation, when that observation is not used to fit the model: $e_{[i]} = y_i - \hat{y}_{[i]}$
- We define the **leave-one-out cross-validated RMSE** to be

$$\text{RMSE}_{\text{LOOCV}} = \sqrt{\frac{1}{n}\sum_{i=1}^{n} e_{[i]}^2}.$$

- LOOCV gives a measure of the possible out-of-sample prediction error
- In general, to perform this calculation, we would be required to fit the model n times, once with each possible observation removed.

Validation-based selection

- However, for leave-one-out cross-validation and linear models, the equation can be rewritten as

$$\text{RMSE}_{\text{LOOCV}} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(\frac{e_i}{1-h_{ii}}\right)^2},$$

where h_{ii} are the **leverages** the diagonal elements of the (projection) matrix

$$H = X(X^\top X)^{-1}X^\top$$

and e_i are the usual residuals.

- This is great, because now we can obtain the RMSE_{LOOCV} by fitting only one model!

Validation-based selection

- In practice 5 or 10 fold cross-validation are much more popular. For example, in 5-fold cross-validation, the model is fit 5 times, each time leaving out a fifth of the data, then predicting on those values.
- More on cross-validation to in a master course of statistical learning and simply use LOOCV here
- Let's calculate RMSE_{LOOCV} for some of the previous models
- We first write a function which calculates the LOOCV RMSE as defined using the shortcut formula for linear models.

```
calc_loocv_rmse = function(model) {
  sqrt(mean((resid(model) / (1 - hatvalues(model))) ^ 2))
  }
calc_loocv_rmse(lm(flipper_length_mm ~ 1, data = penguins)) # no variable
```

```
[1] 14.03686
```

Validation-based selection

```
calc_loocv_rmse(lm(flipper_length_mm ~
      body_mass_g + bill_length_mm, data = penguins)) # some variables
```

```
[1] 6.450086
```

```
calc_loocv_rmse(lm(flipper_length_mm ~ body_mass_g + bill_length_mm +
      bill_depth_mm + year, data = penguins)) # model with all variables
```

```
[1] 5.583364
```

- The model selected by stepwise selection also has the lowest RMSE_{LOOCV} (this is not always the case).

Categorical Predictors and Interactions [3]

[3] Material in these slides was heavily influenced by David Dalpiaz *Applied Statistics with R!*, the book is under active development.

Introductory example

- Let's have a closer look at the `penguins` dataset, we haven't used many of the variables yet:

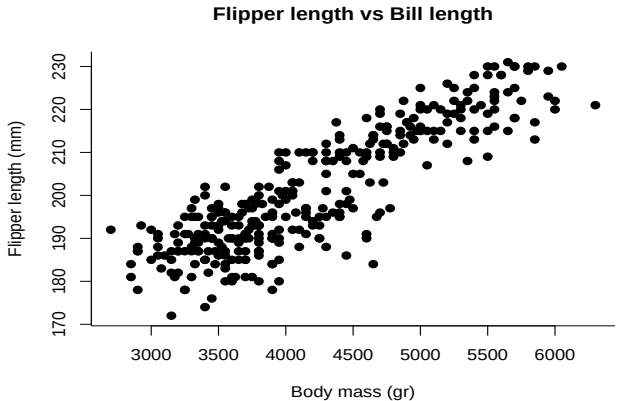
```
head(penguins)
```

```
  species    island bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
1  Adelie Torgersen           39.1          18.7               181        3750
2  Adelie Torgersen           39.5          17.4               186        3800
3  Adelie Torgersen           40.3          18.0               195        3250
5  Adelie Torgersen           36.7          19.3               193        3450
6  Adelie Torgersen           39.3          20.6               190        3650
7  Adelie Torgersen           38.9          17.8               181        3625
     sex year
1   male 2007
2 female 2007
3 female 2007
5 female 2007
6   male 2007
7 female 2007
```

We wish to see whether other properties of a penguin (the species, the sex, the island) affect the flipper's length (Y).

Introductory example

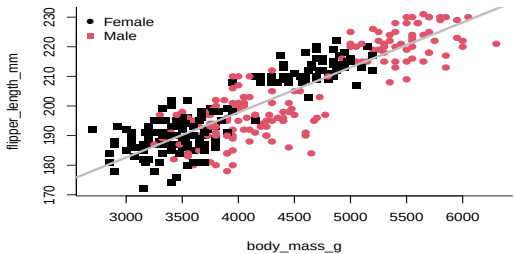
- We will start by plotting the data, let's look at body mass length and flipper's length:



Introductory example

- We now fit the simple linear regression model: $Y = \beta_0 + \beta_1 x_1 + \varepsilon$

```
f0_slr <-lm(flipper_length_mm ~ body_mass_g , data = penguins)
```



- We notice some systematic deviations for males and females (and some other structure, more on this later).

Dummy Variables

- A new model

$$Y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \varepsilon,$$

where x_1 and Y remain the same, but now

$$x_2 = \begin{cases} 1 & \text{male} \\ 0 & \text{female} \end{cases}$$

- We call x_2 a **dummy variable**

A dummy variable is a numerical variable that is used in a regression analysis to code for a binary categorical variable.

Dummy Variables

- Since x_2 can only take values 0 and 1, we can effectively write two different models, one for males and one for females.
  1. For female penguins, that is $x_2 = 0$, we have,

$$Y = \beta_0 + \beta_1 x_1 + \varepsilon.$$

  2. For male penguins, that is $x_2 = 1$, we have,

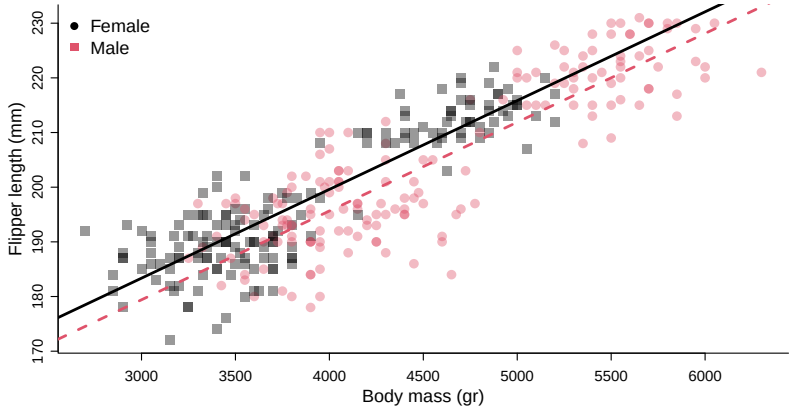
$$Y = (\beta_0 + \beta_2) + \beta_1 x_1 + \varepsilon.$$

- Notice that these models share the same slope, β_1, but have different intercepts, differing by β_2.
- We fit the model

```
fc_mlr_add <- lm(flipper_length_mm ~ body_mass_g + sex, data = penguins)
```

and we plot the *two* fitted model

Dummy Variables



- Not quite two different models: σ is "shared" by both groups and estimated using all data.

Dummy Variables

- We would like to test:

$$\mathrm{H}_0 : \beta_2 = 0 \quad \text{vs} \quad \mathrm{H}_A : \beta_2 \neq 0.$$

- The test statistic and p-value for the t-test

```
summary(fc_mlr_add)$coefficients["sexmale",]
```

```
      Estimate       Std. Error          t value         Pr(>|t|)
-3.956995825081  0.801177208007 -4.938977027226  0.000001250047
```

- The F test

```
anova(f0_slr, fc_mlr_add)
```

```
Analysis of Variance Table

Model 1: flipper_length_mm ~ body_mass_g
Model 2: flipper_length_mm ~ body_mass_g + sex
  Res.Df   RSS Df Sum of Sq      F     Pr(>F)
1    331 15516
2    330 14448  1      1068 24.393 0.00000125 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

- Notice that the F test statistic is the t test statistic squared (we don't discuss why).

Interactions

- Now consider the following model,

$$Y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_1 x_2 + \varepsilon,$$

  where x_1, x_2, and Y are the same as before, but we have added a new **interaction** term $x_1 x_2$ which multiplies x_1 and x_2, so we also have an additional β parameter β_3.
- This model essentially creates two slopes and two intercepts, β_2 being the difference in intercepts and β_3 being the difference in slopes.
  - For $x_2 = 0$ we have
$$Y = \beta_0 + \beta_1 x_1 + \varepsilon$$
  - For $x_2 = 1$ we have
$$Y = (\beta_0 + \beta_2) + (\beta_1 + \beta_3) x_1 + \varepsilon$$

Interactions

- How do we fit this model in R?
  1. Create a new variable, then fit a model like any other.

     ```
     penguins$sexInt <- penguins$body_mass_g * (penguins$sex == "Male")
     do_not_do_this <- lm(flipper_length_mm ~ body_mass_g + sex + sexInt,
                          data = penguins)
     ```

     You should only do this as a last resort!
  2. Use the existing data with an interaction term such as the `:` operator.

     ```
     fi_mlr_int <- lm(flipper_length_mm ~ body_mass_g + sex + body_mass_g : sex,
                      data = penguins)
     ```
  3. An alternative method uses the `*` operator. This method automatically creates the interaction term, as well as any "lower order terms" which in this case are the first order terms for `body_mass_g` and `sex`

     ```
     fi_mlr_int2 <-lm(flipper_length_mm ~ body_mass_g * sex, data = penguins)
     ```

Interactions

```
summary(fi_mlr_int)
```

```
Call:
lm(formula = flipper_length_mm ~ body_mass_g + sex + body_mass_g:sex,
    data = penguins)

Residuals:
     Min       1Q   Median       3Q      Max
-22.1736  -4.5905   0.6326   4.6430  15.0168

Coefficients:
                     Estimate  Std. Error t value Pr(>|t|)
(Intercept)         133.2351665   3.0424512  43.792   <2e-16 ***
body_mass_g           0.0166038   0.0007763  21.387   <2e-16 ***
sexmale              -1.3972849   4.2739522  -0.327    0.744
body_mass_g:sexmale  -0.0006177   0.0010130  -0.610    0.542
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 6.623 on 329 degrees of freedom
```

Interactions

```
Multiple R-squared:  0.7787,        Adjusted R-squared:  0.7767
F-statistic: 385.9 on 3 and 329 DF,  p-value: < 2.2e-16
```

- In this case, testing for $\beta_3 = 0$ is testing for two lines with parallel slopes versus two lines with possibly different slopes.

$$H_0 : \beta_3 = 0 \qquad VS \qquad H_1 : \beta_3 \neq 0$$

The `body_mass_g:sexmale` line in the `summary` output uses a t-test to perform the test.

```
summary(fi_mlr_int)$coefficients["body_mass_g:sexmale",]
```

```
     Estimate    Std. Error       t value      Pr(>|t|)
-0.0006176503  0.0010129746 -0.6097391962  0.5424554613
```

Interactions

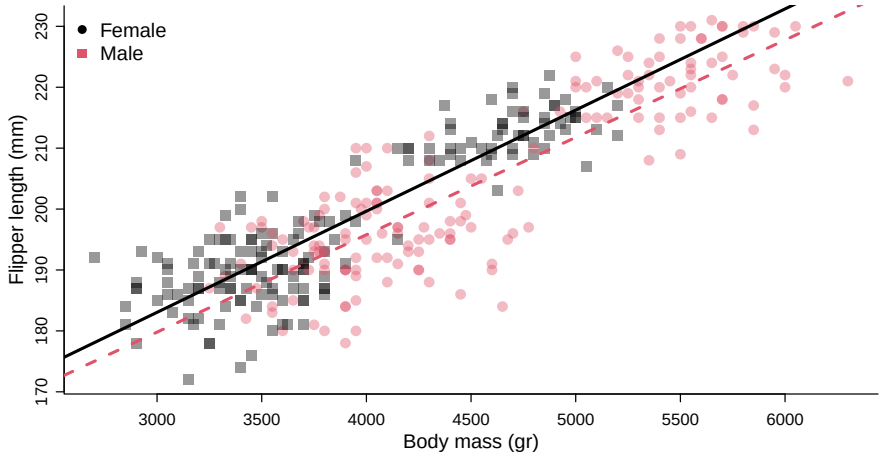
- We could also use an ANOVA F-test: the additive model (null model) against the interaction model (the alternative model).

```
anova(fc_mlr_add, fi_mlr_int)
```

```
Analysis of Variance Table

Model 1: flipper_length_mm ~ body_mass_g + sex
Model 2: flipper_length_mm ~ body_mass_g + sex + body_mass_g:sex
  Res.Df   RSS Df Sum of Sq      F Pr(>F)
1    330 14448
2    329 14432  1    16.308 0.3718 0.5425
```

- We see this test has the same p-value as the t-test. Also the p-value is quite high, so between the two, we choose the additive model.

Interactions



Factor Variables

- The `sex` variable is a string

```
class(penguins$sex)
```

```
[1] "character"
```

- How can `R` include a string into a model? It makes the variable into a numeric.
- We could do the same and create a variable `gender` which takes value `0` for male penguins and `1` for female penguins:

```
penguins$gender <- 0
penguins$gender[penguins$sex == "female"] = 1
penguins$gender[1:4]
```

```
[1] 0 1 1 1
```

```
class(penguins$gender)
```

```
[1] "numeric"
```

Factor Variables

- We use this new variable in a model similar to `fc_mlr_add`

```
fc_mlr_add_alt <- lm(flipper_length_mm ~ body_mass_g + gender,
                     data = penguins)
```

- We have the same fitting

```
summary(fc_mlr_add)$r.squared; summary(fc_mlr_add_alt)$r.squared
```

```
[1] 0.7784678
```

```
[1] 0.7784678
```

- However it seems that it doesn't produce the same results.

```
coef(fc_mlr_add)
```

```
 (Intercept)  body_mass_g      sexmale
134.63634714   0.01624103 -3.95699583
```

```
coef(fc_mlr_add_alt)
```

```
 (Intercept)  body_mass_g       gender
130.67935131   0.01624103   3.95699583
```

Factor Variables

- Right away we notice that the intercept is different, as is the the coefficient in front of `sex`. We also notice that the remaining two coefficients are of the same magnitude as their respective counterparts using the `sex` variable, but with a different sign.
- What is happening?
- When we ask `R` to use a variable stored as a string, the program turns this into a `factor` and from the factor it creates dummy variables: one dummy for each level *except the reference level*.
- The reference level is taken to be the first level, by the order is set alphabetically.
- **factor** variables are special variables that `R` uses to deal with categorical variables.

Factor Variables

- By having the program create the levels we can avoid doing unreasonable things such as making prediction for levels we have not observed:

```
predict(fc_mlr_add_alt,newdata = data.frame(body_mass_g=45,gender=0.2))
       1
132.2016
##a useless prediction - what does gender=0.2 mean???
predict(fc_mlr_add,newdata=data.frame(body_mass_g=45,sex="unknown"))
## this gives an error
predict(fc_mlr_add,newdata=data.frame(body_mass_g=c(45,45),sex=c("male",'
       1        2
131.4102 135.3672
```

Factor Variables

- Under the hood, `R` has turned the string into a dummy variable, with `female` as the baseline ($x_2 = 0$):

```
head(model.matrix(fc_mlr_add))
```

```
  (Intercept) body_mass_g sexmale
1           1        3750       1
2           1        3800       0
3           1        3250       0
5           1        3450       0
6           1        3650       1
7           1        3625       0
```

- `factor` variables are the way in which R stores categorical variable

Factor Variables

- Factor variables have **levels** which are the possible values (categories) that the variable may take, in this case male or female

```
penguins$sex2 <- factor(penguins$sex)
penguins$sex2[1:4]
```

```
[1] male   female female female
Levels: female male
```

```
levels(penguins$sex2)
```

```
[1] "female" "male"
```

```
coef(lm(flipper_length_mm ~ body_mass_g + sex2, data = penguins))
```

```
 (Intercept)  body_mass_g      sex2male
134.63634714   0.01624103  -3.95699583
```

- `factor` can also contain ordinal variables - see ?factor to know more

Factors with More Than Two Levels

- Let's now consider a factor variable with more than two levels. `species` is an example.

```
penguins$species <- factor(penguins$species)
table(penguins$species)
```

```
   Adelie Chinstrap    Gentoo
      146        68       119
```

```
unique(as.numeric(penguins$species))
```

```
[1] 1 3 2
```

- Let's define three dummy variables related to the `species` factor variable.

$$v_1 = \begin{cases} 1 & \text{Adelie} \\ 0 & \text{not Adelie} \end{cases} \qquad v_2 = \begin{cases} 1 & \text{Chinstrap} \\ 0 & \text{not Chinstrap} \end{cases} \qquad v_3 = \begin{cases} 1 & \text{Gentoo} \\ 0 & \text{not Gentoo} \end{cases}$$

Factors with More Than Two Levels

- We fit an additive model in `R`, using `flipper_length_mm` as the response, and `body_mass_g` and `species` as predictors that uses "three regression lines" to model flipper's length, one for each of the possible `species` levels.

$$Y = \beta_0 + \beta_1 x + \beta_2 v_2 + \beta_3 v_3 + \varepsilon,$$

  where
  - Y is the flipper's length
  - x is `body_mass_g`, the displacement in cubic inches,
  - v_2 and v_3 are the dummy variables define above.
- Notice we use two dummy variables to codify the three level categorical variable.

Factors with More Than Two Levels

- In R

```
fc_mass_species <- lm(flipper_length_mm ~ body_mass_g + species, data = penguins
fc_mass_species
```

```
Call:
lm(formula = flipper_length_mm ~ body_mass_g + species, data = penguins)

Coefficients:
     (Intercept)       body_mass_g  speciesChinstrap     speciesGentoo
      158.546071          0.008515          5.491543         15.328938
```

- R doesn't use v_1 because it doesn't need to.
- To create three lines, it only needs two dummy variables since it is using a **reference level**.

Factors with More Than Two Levels

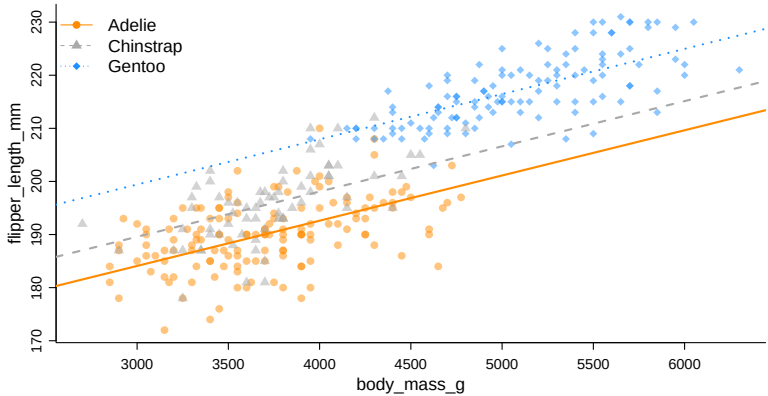
- R automatically creates an appropriate model matrix:

```
model.matrix(fc_mass_species)[c(1,220,330),]
    (Intercept) body_mass_g speciesChinstrap speciesGentoo
1             1        3750                0             0
228           1        5800                0             1
341           1        3400                1             0
colSums(model.matrix(fc_mass_species)[,3:4])
speciesChinstrap    speciesGentoo
              68              119
table(penguins$species)
   Adelie Chinstrap    Gentoo
      146        68       119
```

Factors with More Than Two Levels

- The three "sub models" are then:
  - Adelie: $Y = \beta_0 + \beta_1 x + \varepsilon$
  - Chinstrap: $Y = (\beta_0 + \beta_2) + \beta_1 x + \varepsilon$
  - Gentoo: $Y = (\beta_0 + \beta_3) + \beta_1 x + \varepsilon$
- Notice that they all have the same slope. However, using two dummy variables, we achieve three intercepts.
- In this case Adelie is the **reference level**, β_0 is specific to Adelie, but β_2 and β_3 are used to represent quantities relative to Adelie

Factors with More Than Two Levels



- Is the relationship between body mass and flipper length the same for all species?

Factors with More Than Two Levels

- We can assess this using an interaction model:

```
fc_mass_int_species <- lm(flipper_length_mm ~ body_mass_g * species, data = penguins)
fc_mass_int_species
```

```
Call:
lm(formula = flipper_length_mm ~ body_mass_g * species, data = penguins)

Coefficients:
                  (Intercept)                   body_mass_g
                   165.603241                      0.006610
            speciesChinstrap                 speciesGentoo
                   -14.222367                      4.063979
body_mass_g:speciesChinstrap     body_mass_g:speciesGentoo
                     0.005295                      0.002730
```

- R has fit the model

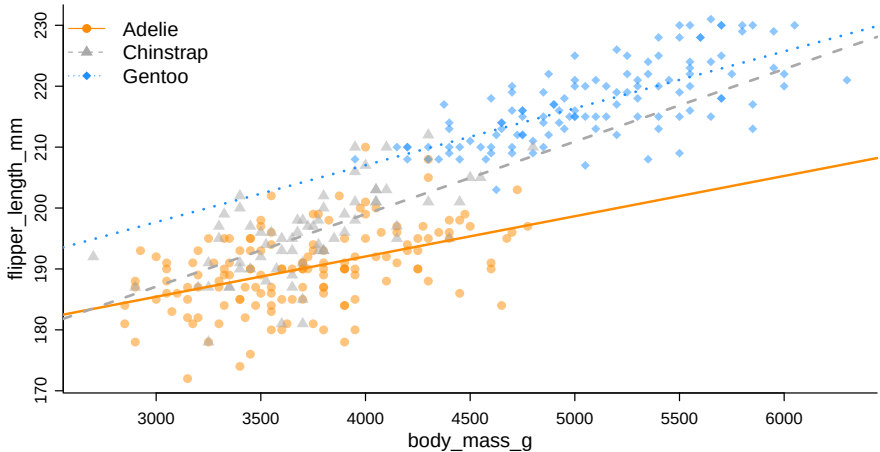
$$Y = \beta_0 + \beta_1 x + \beta_2 v_2 + \beta_3 v_3 + \gamma_2 x v_2 + \gamma_3 x v_3 + \varepsilon$$

(We're using γ like a β parameter for simplicity, so that, for example β_2 and γ_2 are both associated with v_2)

Factors with More Than Two Levels

- Now, the three "sub models" are:
  - Adelie: $Y = \beta_0 + \beta_1 x + \varepsilon$.
  - Chinstrap: $Y = (\beta_0 + \beta_2) + (\beta_1 + \gamma_2)x + \varepsilon$.
  - Gentoo: $Y = (\beta_0 + \beta_3) + (\beta_1 + \gamma_3)x + \varepsilon$.
- Interpretation of some parameters and coefficients:
  - $(\hat{\beta}_0 + \hat{\beta}_2) = 165.603 + -14.222 = 151.381$ is the estimated average flipper length of a Chinstrap penguin weighting 0 gr.
  - $(\hat{\beta}_1 + \hat{\gamma}_3) = 0.007 + 0.003 = 0.009$ is the estimated change in average flipper length of Gentoo penguins whose weight differs of 1 gr.
- So, as we have seen before β_2 and β_3 change the intercepts for Chinstrap and Gentoo penguins relative to the reference level of β_0 for Adelie penguins.
- Now, similarly γ_2 and γ_3 change the slopes for Chinstrap and Gentoo penguins relative to the reference level of β_1 for Adelie penguins.

Factors with More Than Two Levels



Factors with More Than Two Levels

- To justify the interaction model (i.e., a unique slope for each `species` level) compared to the additive model (single slope), we can perform an F-test

$$H_0 : \gamma_2 = \gamma_3 = 0$$

  which represents the parallel regression lines we saw before,

$$Y = \beta_0 + \beta_1 x + \beta_2 v_2 + \beta_3 v_3 + \varepsilon.$$

- This can be done using `anova`

Factors with More Than Two Levels

```
anova(fc_mass_species, fc_mass_int_species)
```

```
Analysis of Variance Table

Model 1: flipper_length_mm ~ body_mass_g + species
Model 2: flipper_length_mm ~ body_mass_g * species
  Res.Df    RSS Df Sum of Sq     F  Pr(>F)
1    329 9612.8
2    327 9368.2  2    244.61 4.269 0.01478 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

- We see a low p-value, and thus reject the null. We prefer the interaction model over the additive model.

Linear models repurposed

- What if we only use a factor variable in the analysis? See a factor with two levels:

```
mfa_only <- lm(flipper_length_mm ~ sex, data = penguins)
summary(mfa_only)$coef
```

```
             Estimate Std. Error    t value       Pr(>|t|)
(Intercept) 197.363636   1.056598 186.791565 0.000000000000
sexmale       7.142316   1.487570   4.801332 0.000002391097
```

- The fitted sub model are:
  - female penguins: $Y = \beta_0 + \varepsilon$.
  - male penguins: $Y = (\beta_0 + \beta_1) + \varepsilon$.
- So $E[Y|female] = \beta_0$ and $E[Y|male] = \beta_0 + \beta_1$ and β_1 represents the difference in means and a test with $H_0 : \beta_1 = 0$ can be re-interpreted as a test for $E[Y|female] = E[Y|male]$.
- This is a t-test

Linear models repurposed

- Specifically a t-test in which equal variances are assumed:

```
t.test(flipper_length_mm ~ sex, data = penguins, var.equal = TRUE)
```

```
        Two Sample t-test

data:  flipper_length_mm by sex
t = -4.8013, df = 331, p-value = 0.000002391
alternative hypothesis: true difference in means between group female and group male is not equal to
95 percent confidence interval:
 -10.068599  -4.216033
sample estimates:
mean in group female   mean in group male
            197.3636             204.5060
```

```
summary(mfa_only)$coef
```

```
               Estimate Std. Error    t value       Pr(>|t|)
(Intercept) 197.363636   1.056598 186.791565 0.000000000000
sexmale       7.142316   1.487570   4.801332 0.000002391097
```

Linear models repurposed

- We get the same confidence interval for the difference in the two means:

```
confint(mfa_only)[2,]
```

```
   2.5 %    97.5 %
4.216033 10.068599
```

```
t.test(flipper_length_mm ~ sex, data = penguins,var.equal = TRUE)$conf.in
```

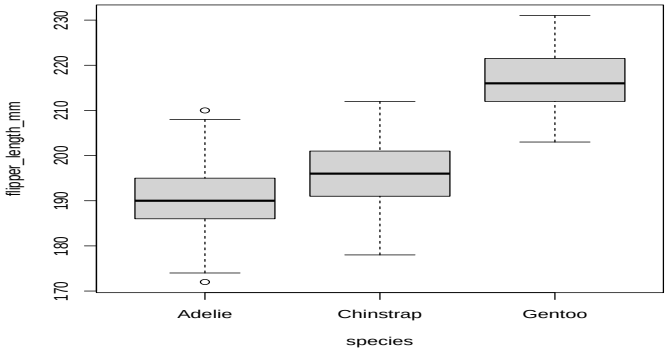
```
[1] -10.068599  -4.216033
attr(,"conf.level")
[1] 0.95
```

- In this case we have evidence to say the two means are different

Linear models repurposed

- Visually:

```
boxplot(flipper_length_mm ~ species, data = penguins)
```



Linear models repurposed

- What happens to factors with more than levels:

```
summary(lm(flipper_length_mm ~ species, data = penguins))$coef
```

```
                  Estimate Std. Error   t value      Pr(>|t|)
(Intercept)      190.10274  0.5522277 344.24702  0.000000e+00
speciesChinstrap   5.72079  0.9796493   5.83963  1.252748e-08
speciesGentoo     27.13255  0.8240767  32.92479 2.680334e-106
```

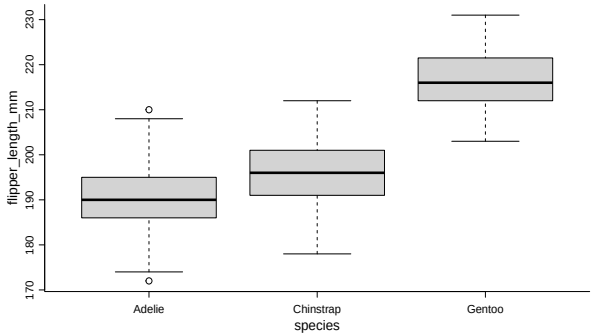
Linear models repurposed

- There a widely-employed statistical technique called ANOVA which generalizes the T-test.

```
 summary(aov(flipper_length_mm ~ species, data = penguins))
             Df Sum Sq Mean Sq F value Pr(>F)
species       2  50526   25263   567.4 <2e-16 ***
Residuals   330  14693      45
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
 summary(lm(flipper_length_mm ~ species, data = penguins))$fstatistic
  value   numdf   dendf
567.407   2.000 330.000
```

Linear models repurposed

- Visually:



- Notice that equal variances are assumed - this might be not an obvious assumption in some situations.

Model checking [4]

[4] Material in these slides was heavily influenced by David Dalpiaz *Applied Statistics with R!*, the book is under active development.

Model assumptions

- The least square estimate is "optimal" if the relationship between Y and $(X_1, \ldots, X_p)$ is approximately linear.
- We have discussed methods to test for significance and for estimating the variability of estimates/predictions which is based on the assumption of iid normal errors.
- This is typically expressed as:

$$Y_i \sim N(\beta_0 + \beta_1 x_{1,i} + \ldots + \beta_p x_{p,i}, \sigma) \text{ for each } i,$$

which can be rewritten as

$$\varepsilon_i = (Y_i - (\beta_0 + \beta_1 x_{1,i} + \ldots + \beta_p x_{p,i})) \sim N(0, \sigma) \text{ for each } i.$$

- If the assumptions are not valid we can not rely on the theory to do inference.

Model assumptions

- Often, the assumptions of linear regression, are stated as:
  Linearity: the response can be written as a linear combination of the predictors. (With noise about this true linear relationship.)
  Independence: the errors are independent.
  Normality: the distribution of the errors should follow a normal distribution.
  Equal Variance: the error variance is the same at any set of predictor values.
- There are a number of statistical tests and graphical approaches to verify the validity of these assumptions.
- Notice that other things can go wrong and make the model not valid: statistical modeling is a craft more than a science.

Residuals-based displays

- We define the residuals r_i (often indicated also as e_i) which are a sample estimate of the errors ε_i:

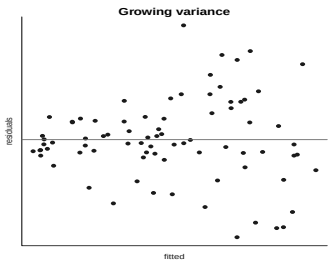
$$r_i = (y_i - \hat{y}_i)$$

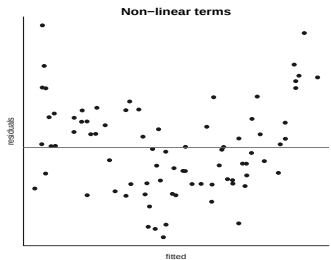
- By definition: $\sum r_i/n = 0$ and $\hat{\sigma} = \sum r_i^2/(n-p)$.
- We use the residuals to verify whether assumptions are met (like for simple linear models, but now its harder to separate out the effect of each variable on the quality of the fit)
- We use residual plots to check both the linearity and constant variance assumptions.
- We use the qqplot of residuals to verify the normality assumption.
- We can use residuals to verify the independence assumption - but we should also control for this when designing the data collection.

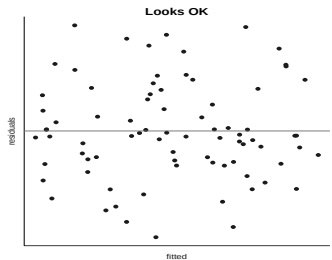
Residuals-based displays

- If the model is well specified, the sample of $(r_1, \ldots, r_n)$ should be iid normally distributed with a constant variance.
- For each model we fit to a dataset we have a different vector of residuals r_i
- First residual plot: plot r_i against $\hat{y}_i$. There should be not discernible pattern in the data (no systematic under- or over-estimation) and constant variance.
- If the variance of the data is constant there should be a scatter around the mean (which is 0)
- If the linear terms capture the entire relationship between X and Y, r_i there should be no systematic under-or over-estimation for some values of X.
- Looking at the plots can give some indication on the possible solution to the problem.

Residuals-based displays







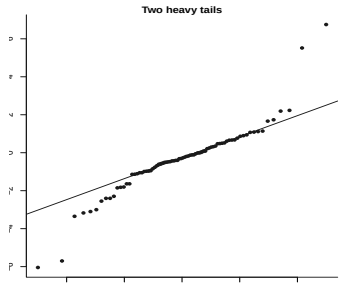
QQplots

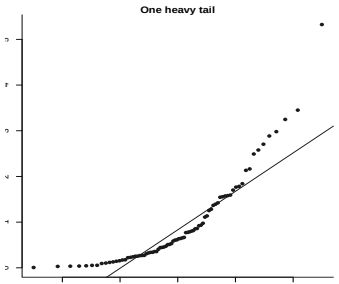
- The residuals should be normally distributed.
- If that was true the empirical cdf/pdf should look like the cdf/pdf of a normal. We compare them.
- Option 1: plot a histogram of residuals.
- Option 2 (a better option): compare empirical and theoretical quantiles via a qqplot.
- Can help identify long tails (excessive variance)
- Sometimes for qqplot and summary statistics it is easier to use the standardized residuals (see `rstandard`)

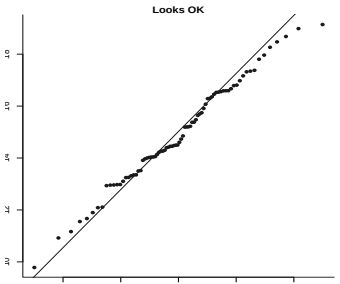
$$\frac{y_i - \hat{y}_i}{\hat{\sigma}}$$

which should follow a standard normal. This means for example that 95% of the residuals should have values within (-2,2).

QQplots







QQplots

- `qqplot` and `qqline` in `R`
- See also the `qqPlot` function in the `car` library
- Sort the residuals from the smallest $r_{(1)}$ to the largest $r_{(n)}$
- Assign to the observation in position i the empirical cdf value, for example $(i - 0.5)/n$.
- Compare the value of $r_{(i)}$ to the theoretical value of a normal sample of size n.
- Useful to identify specif points with particularly high residuals.
- Useful to identify deviations from normality

Model checking

- If model assumptions are not valid the inference might be dubious.
- But nothing is set in stone: models which deviate from the assumptions can be still be OK.
- Nevertheless if keeping a non-significance variable in the model improves the model-checks it might be a good reason to keep the variable.
- The same hold for transformations discussed below.
- Model building is an iterative process: check the data, fit a model or two, check residuals and iterate.
- Some subjective decisions on what is a "good fit"
- In R `plot(lm.object)` gives some useful plots for model-checking.
- There can be other problematic cases which we do not discuss - it is a wide field of research.

Transformations [5]

[5] Material in these slides was heavily influenced by David Dalpiaz *Applied Statistics with R!*, the book is under active development.

Transformations

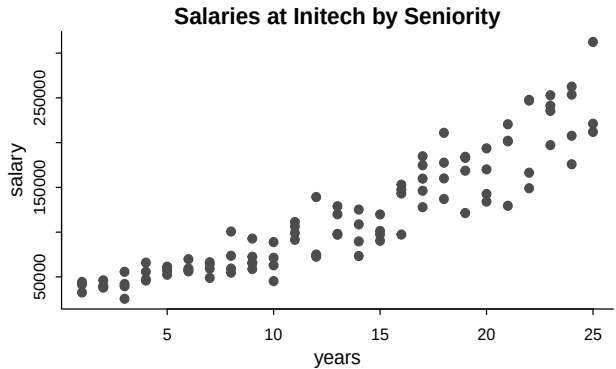
All happy families are alike; each unhappy family is unhappy in its own way. (Leo Tolstoy)

Models can be problematic in many different ways...

- Model checks can highlight issues with linearity and homoschedasticity assumptions (among others)
- Assumption is: **linear** relationship between X and Y
- Sometimes the relationship between X and Y is strong but not linear: one possible approach is to transform X or Y to make the relationship linear
- Assumption is: **constant variance**
- Transform Y to make the variance more constant

Response Transformation

- Some fictional salary data from the fictional company Initech. We will try to model `salary` as a function of years of experience.



Response Transformation

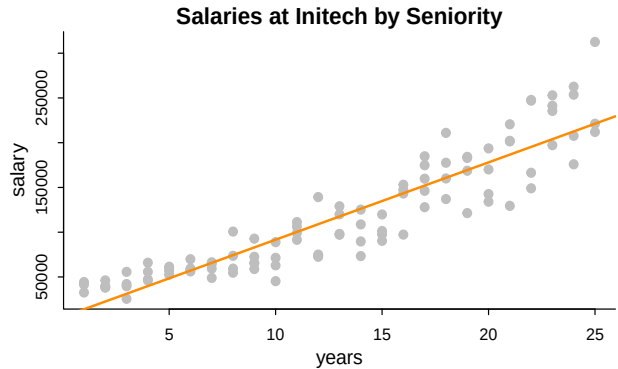
- We first fit a simple linear model.

```
initech_fit <- lm(salary ~ years, data = initech)
summary(initech_fit)
```

```
Call:
lm(formula = salary ~ years, data = initech)

Residuals:
   Min     1Q Median     3Q    Max
-57225 -18104    241  15589  91332

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)     5302       5750   0.922    0.359
years           8637        389  22.200   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 27360 on 98 degrees of freedom
Multiple R-squared:  0.8341,        Adjusted R-squared:  0.8324
F-statistic: 492.8 on 1 and 98 DF,  p-value: < 2.2e-16
```

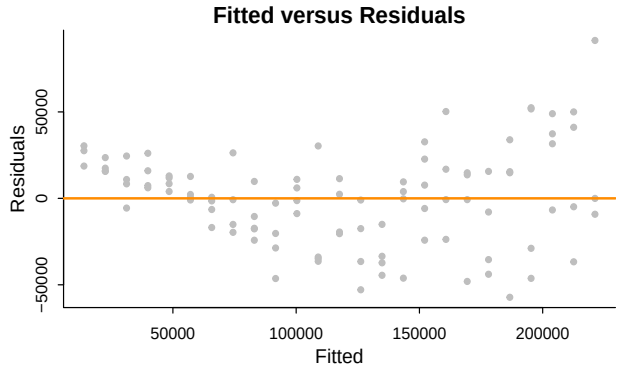
Response Transformation

- Does it meet the model assumptions?



Looks like a decent fit.

Response Transformation



- However, from the fitted versus residuals plot it appears there is non-constant variance. Specifically, the variance increases as the fitted value increases.

Variance stabilizing transformations

- Under usual assumptions
$$\text{Var}[Y|X = x] = \sigma^2$$
which is a constant value for any value of x.
- However, here we see that the variance is a function of the mean,
$$\text{Var}[Y \mid X = x] = h(\text{E}[Y \mid X = x]).$$
In this case, h is some increasing function.
- In order to correct for this, we would like to find some function of Y, $g(Y)$ (**variance stabilizing transformation**) such that,
$$\text{Var}[g(Y) \mid X = x] = c$$
where c is a constant that does not depend on the mean, $\text{E}[Y \mid X = x]$.

Variance stabilizing transformations

- A common variance stabilizing transformation when we see increasing variance in a fitted versus residuals plot is $\log(Y)$.
- Also, if the values of a variable range over more than one order of magnitude and the variable is **strictly positive**, then replacing the variable by its logarithm is likely to be helpful.

$$\log(Y_i) = \beta_0 + \beta_1 x_i + \varepsilon_i.$$

- In the original scale of the data we have

$$Y_i = \exp(\beta_0 + \beta_1 x_i) \cdot \exp(\varepsilon_i)$$

  which has the errors entering the model in a multiplicative fashion.
- We turn the additive model into a *multiplicative* model

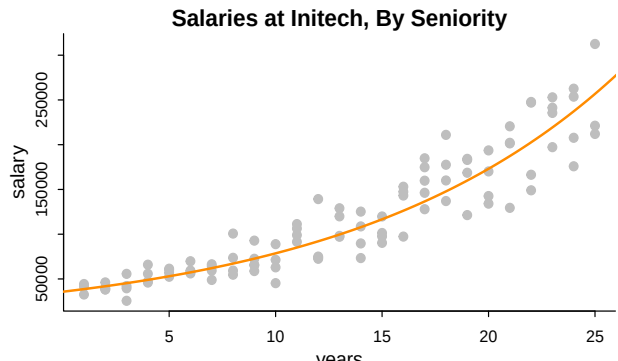
Variance stabilizing transformations

- Fitting this model in R requires only a minor modification to our formula specification.

```
initech_fit_log <- lm(log(salary) ~ years, data = initech)
```

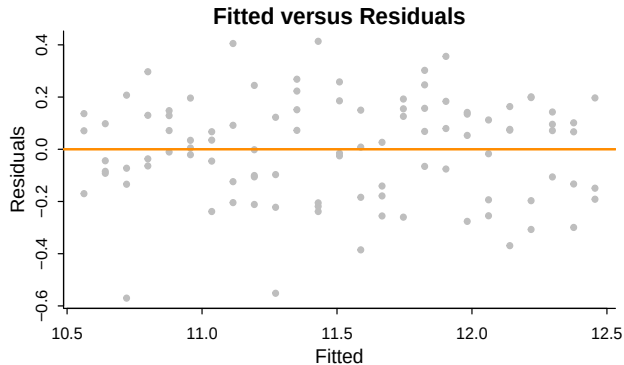
Variance stabilizing transformations

- On the original scale



Variance stabilizing transformations

- We check the residuals



- The fitted versus residuals plot looks much better. It appears the constant variance assumption is no longer violated.

Variance stabilizing transformations

- Let's compare errors

```
sqrt(mean(resid(initech_fit) ^ 2))
```

```
[1] 27080.16
```

```
sqrt(mean(resid(initech_fit_log) ^ 2))
```

```
[1] 0.1934907
```

But wait, that isn't fair, this difference is simply due to the different scales being used.

```
sqrt(mean((initech$salary - fitted(initech_fit)) ^ 2))
```

```
[1] 27080.16
```

```
sqrt(mean((initech$salary - exp(fitted(initech_fit_log))) ^ 2))
```

```
[1] 24280.36
```

Variance stabilizing transformations

- The transformed response is a **linear** combination of the predictors,

$$\log(\hat{y}(x)) = \hat{\beta}_0 + \hat{\beta}_1 x = 10.484 + 0.079x.$$

- If we re-scale the data from a log scale back to the original scale of the data, we now have

$$\hat{y}(x) = \exp(\hat{\beta}_0)\exp(\hat{\beta}_1\, x) = \exp(10.484)\exp(0.079\, x).$$

- Comment: average salary increases $\exp(0.079) = 1.0822$ times for one additional year of experience.
- Comparing the RMSE using the original and transformed response, we also see that the log transformed model simply fits better, with a smaller average squared error.
- Issue: from probability we know that $E[g(X)] \neq g(E[X])$: we are modeling a different variable, back-transforming needs to be done with care. [Notice that instead the median$(\log(X)) = \log(\text{median}(X))$.]

Box-Cox transform

Box-Cox transform

$$y_\lambda = \begin{cases} \frac{y^\lambda - 1}{\lambda} & \lambda \neq 0 \\ \log(y) & \lambda = 0 \end{cases}$$

- Transformation is defined for positive data only.
- Possible remedy: add a constant to the data.
- Choose: $\lambda < 1$ for positively skewed data, $\lambda > 1$ for negatively skewed data.
- The λ parameter is chosen by numerically maximizing the log-likelihood,

$$L(\lambda) = -\frac{n}{2}\log(SS_{\text{err}}(\lambda)/n) + (\lambda - 1)\sum_{i=1}^{n}\log(y_i).$$

where $SS_{\text{err}}(\lambda) = \sum_{i=1}^{n}(y_{\lambda,i} - \hat{y}_{\lambda,i})^2$

Box-Cox transform

- Procedure is implemented in the package `MASS`

  ```
  library(MASS)
  ```

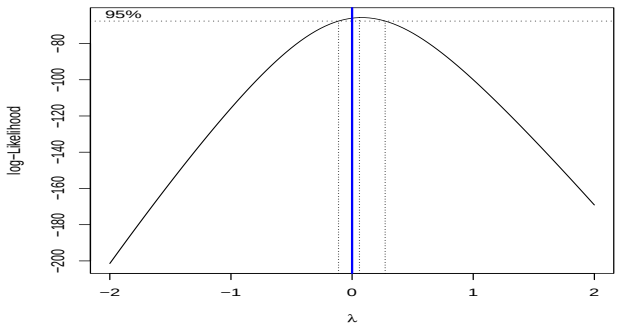
- We then use the `boxcox` function to find the best transformation of the form considered by the Box-Cox method.
- A $100(1-\alpha)\%$ confidence interval for λ is,

$$\left\{\lambda : L(\lambda) > L(\hat{\lambda}) - \frac{1}{2}\chi^2_{1,\alpha}\right\}$$

  which `R` will plot for us to help quickly select an appropriate λ value.

Box-Cox transform

```
boxcox(initech_fit, plotit = TRUE)
```



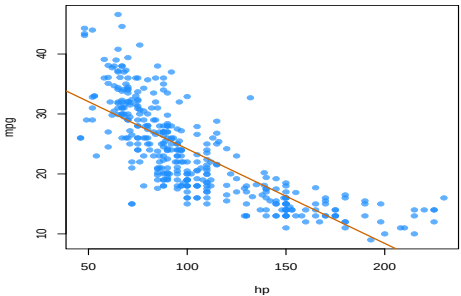
- We often choose a "nice" value from within the confidence interval, instead of the value of λ, $\hat{\lambda} = 0.061$, that truly maximizes the likelihood. In this case we choose $\lambda = 0$.

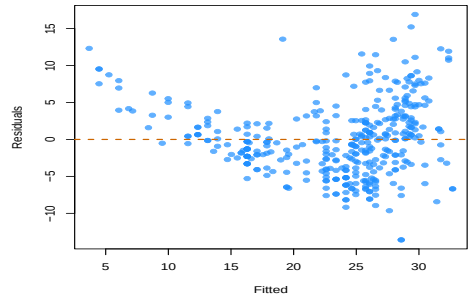
Transformations of predictor variables.

- The linear model is termed **linear** not because the regression curve is a plane, but because **the effects of the parameters are linear**.
- The following models can be transformed into simple linear models:
  - $Y = \beta_0 + \beta_1 x^2 + \varepsilon$
  - $Y = \beta_0 + \beta_1 \log(x) + \varepsilon$
  - $Y = \beta_0 + \beta_1 (x^3 - \log(|x|) + 2^x) + \varepsilon$
- Rather than working with the sample $(x_1, y_1), \ldots, (x_n, y_n)$, we consider the transformed sample $(\tilde{x}_1, y_1), \ldots, (\tilde{x}_n, y_n)$ with
  - $\tilde{x}_i = x_i^2$, $i = 1, \ldots, n$.
  - $\tilde{x}_i = \log(x_i)$, $i = 1, \ldots, n$.
  - $\tilde{x}_i = x_i^3 - \log(|x_i|) + 2^{x_i}$, $i = 1, \ldots, n$

Transformations of predictor variables.

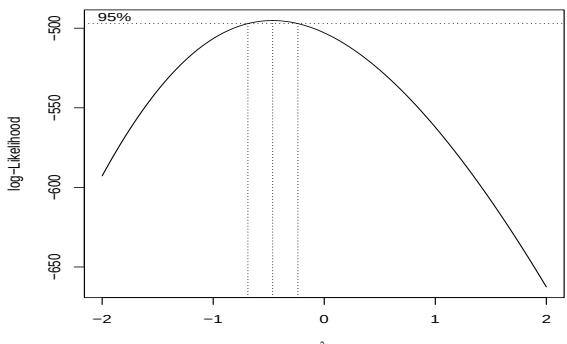
- Sometimes these transformations can help with violation of model assumptions and other times they can be used to simply fit a more flexible model.
- We will attempt to model `mpg` as a function of `hp`.
- We first attempt a SLR, but we see a rather obvious pattern in the fitted versus residuals plot, which includes increasing variance.



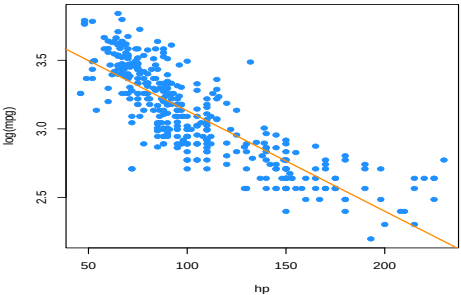


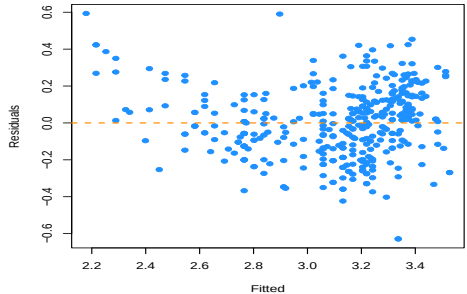
Transformations of predictor variables.

- We attempt a log transform of the response (rough approximation)



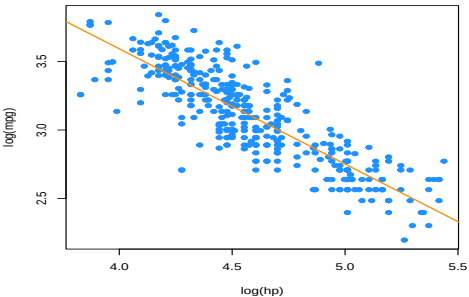
Transformations of predictor variables.

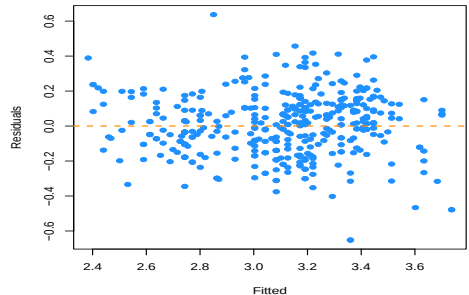




Transformations of predictor variables.

- After performing the log transform of the response, we still have some of the same issues with the fitted versus response. We try also log transforming the predictor.



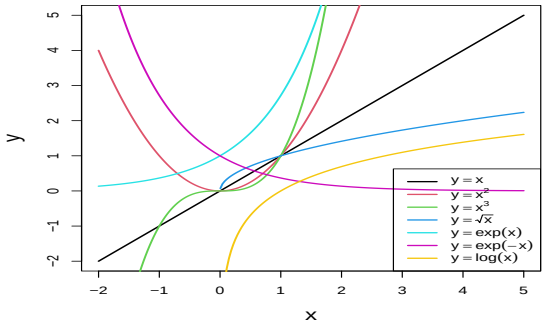


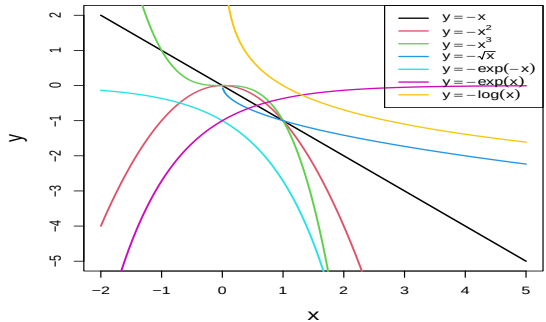
Here, our fitted versus residuals plot looks good.

Tip

- If you apply a nonlinear transformation, namely f, and fit the linear model $Y = \beta_0 + \beta_1 f(x) + \varepsilon$, then there is no point in fit also the model resulting from the negative transformation $-f$. The model with $-f$ is exactly the same as the one with f but with the sign of β_1 flipped!
- As a rule of thumb, use the next figure with the transformations to compare it with the data pattern, then choose the most similar curve, and finally apply the corresponding function with **positive sign**.

Tip


y
x
y = x
y = x²
y = x³
y = √x
y = exp(x)
y = exp(−x)
y = log(x)


y
x
y = −x
y = −x²
y = −x³
y = −√x
y = −exp(−x)
y = −exp(x)
y = −log(x)

Polynomials

- A common "transformation" of a predictor variable is the polynomial transformation. Polynomials are very useful as they allow for more flexible models, but do not change the units of the variables.
- Consider the **nonlinear** transformation, namely f,

$$Y = f(x) + \varepsilon$$

We make no global assumptions about the function f but assume that locally it can be well approximated with a member of a simple class of parametric function, e.g. a constant or straight line.

Polynomials

- Taylor's theorem says that any continuous function can be approximated with polynomial.

Taylor's theorem

Suppose f is a real function on $[a, b]$, $f^{(K-1)}$ is continuous on $[a, b]$, $f^{(K)}(x)$ is bounded for $x \in (a, b)$ then for any distinct points $x_0 < x_1$ in $[a, b]$ there exist a point $\tilde{x}$ between $x_0 < \tilde{x} < x_1$ such that

$$f(x_1) = f(x_0) + \sum_{k=1}^{K-1} \frac{f^{(k)}(x_0)}{k!}(x_1 - x_0)^k + \frac{f^{(K)}(\tilde{x})}{K!}(x_1 - x_0)^K.$$

Notice: if we view $f(x_0) + \sum_{k=1}^{K-1} \frac{f^{(k)}(x_0)}{k!}(x_1 - x_0)^k$ as function of x_1, it's a polynomial in the family of polynomials

$$\mathcal{P}_{K+1} = \{f(x) = a_0 + a_1 x + \cdots + a_K x^K, (a_0, \ldots, a_K)' \in \mathbb{R}^{K+1}\}.$$

Polynomials

- We could fit a polynomial of an arbitrary order K,

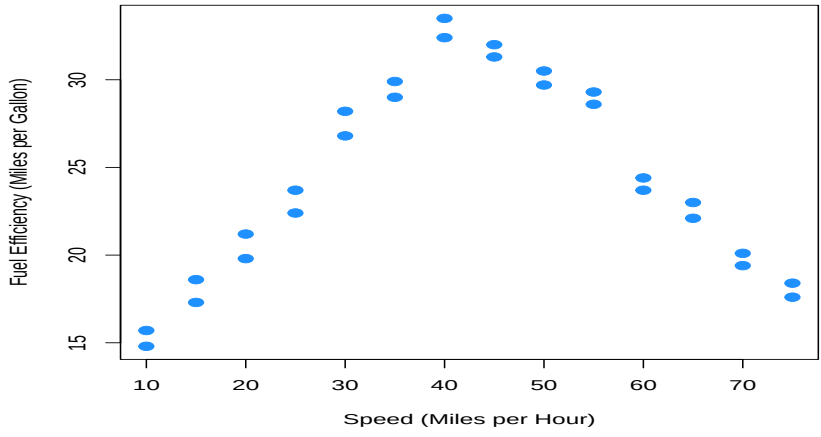
$$Y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \cdots + \beta_K x_i^K + \varepsilon_i$$

and we can think of the polynomial model as the Taylor series expansion of the unknown function.

Polynomials: example

- Suppose you work for an automobile manufacturer which makes a large luxury sedan. You would like to know how the car performs from a fuel efficiency standpoint when it is driven at various speeds.
- Instead of testing the car at every conceivable speed (which would be impossible) you create an experiment where the car is driven at speeds of interest in increments of 5 miles per hour (**Response surface designs**)
- Our goal then, is to fit a model to this data in order to be able to predict fuel efficiency when driving at certain speeds.

Polynomials: example



Polynomials: example

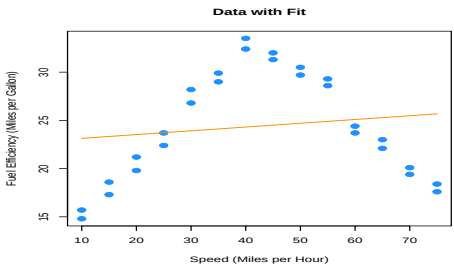
We first fit a simple linear regression to this data.

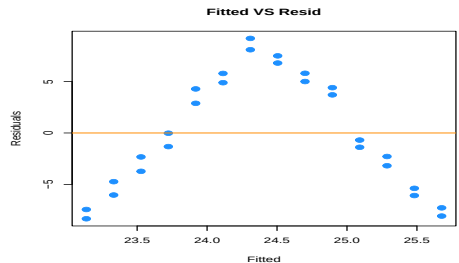
```
econ <- read.csv("data/fuel-econ.csv")
fit1 <- lm(mpg ~ mph, data = econ)
summary(fit1)
```

```
Call:
lm(formula = mpg ~ mph, data = econ)

Residuals:
   Min     1Q Median     3Q    Max
-8.337 -4.895 -1.007  4.914  9.191

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 22.74637    2.49877   9.103 1.45e-09 ***
mph          0.03908    0.05312   0.736    0.469
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 5.666 on 26 degrees of freedom
```

Polynomials: example

```
Multiple R-squared:  0.02039,        Adjusted R-squared:  -0.01729
F-statistic: 0.5411 on 1 and 26 DF,  p-value: 0.4686
```





- Pretty clearly we can do better. Yes fuel efficiency does increase as speed increases, but only up to a certain point.
- We will now add polynomial terms until we fit a suitable fit.

Polynomials: example

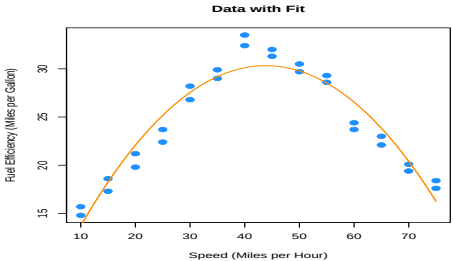
- To add the second order term we need to use the `I()` function in the model specification.
- `I()` is the identity function, which tells `R` "leave this alone".

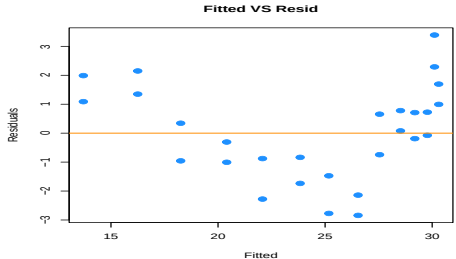
```
fit2 <- lm(mpg ~ mph + I(mph ^ 2), data = econ)
summary(fit2)
```

```
Call:
lm(formula = mpg ~ mph + I(mph^2), data = econ)

Residuals:
    Min      1Q  Median      3Q     Max
-2.8411 -0.9694  0.0017  1.0181  3.3900

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)  2.4444505  1.4241091   1.716   0.0984 .
mph          1.2716937  0.0757321  16.792 3.99e-15 ***
I(mph^2)    -0.0145014  0.0008719 -16.633 4.97e-15 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Polynomials: example

```
Residual standard error: 1.663 on 25 degrees of freedom
Multiple R-squared:  0.9188,        Adjusted R-squared:  0.9123
F-statistic: 141.5 on 2 and 25 DF,  p-value: 2.338e-14
```





Polynomials: example

- While this model clearly fits much better, and the second order term is significant, we still see a pattern in the fitted versus residuals plot which suggests higher order terms will help. Also, we would expect the curve to flatten as speed increases or decreases, not go sharply downward as we see here.

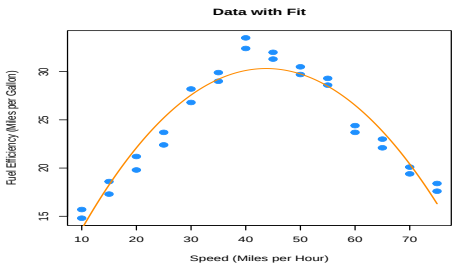
Polynomials: example

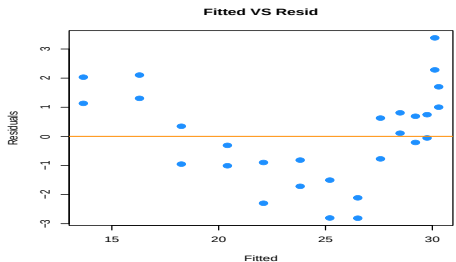
```
fit3 <- lm(mpg ~ mph + I(mph ^ 2) + I(mph ^ 3), data = econ)
summary(fit3)
```

```
Call:
lm(formula = mpg ~ mph + I(mph^2) + I(mph^3), data = econ)

Residuals:
    Min      1Q  Median      3Q     Max
-2.8112 -0.9677  0.0264  1.0345  3.3827

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)  2.258e+00  2.768e+00   0.816   0.4227
mph          1.291e+00  2.529e-01   5.103  3.2e-05 ***
I(mph^2)    -1.502e-02  6.604e-03  -2.274   0.0322 *
I(mph^3)     4.066e-06  5.132e-05   0.079   0.9375
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.697 on 24 degrees of freedom
```

Polynomials: example

```
Multiple R-squared:  0.9188,      Adjusted R-squared:  0.9087
F-statistic: 90.56 on 3 and 24 DF,  p-value: 3.17e-13
```



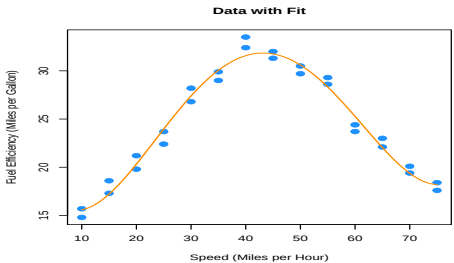


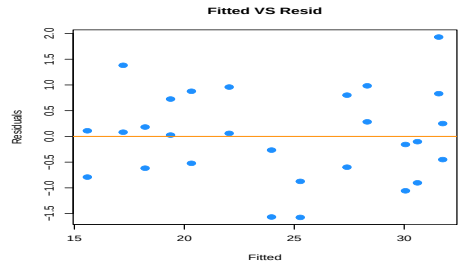
- Adding the third order term doesn't seem to help at all. The fitted curve hardly changes. This makes sense, since what we would like is for the curve to flatten at the extremes.
- For this we will need an **even** degree polynomial term.

Polynomials: example

```
Call:
lm(formula = mpg ~ mph + I(mph^2) + I(mph^3) + I(mph^4), data = econ)

Residuals:
     Min       1Q   Median       3Q      Max
-1.57410 -0.60308  0.04236  0.74481  1.93038

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)  2.146e+01  2.965e+00   7.238 2.28e-07 ***
mph         -1.468e+00  3.913e-01  -3.751  0.00104 **
I(mph^2)     1.081e-01  1.673e-02   6.463 1.35e-06 ***
I(mph^3)    -2.130e-03  2.844e-04  -7.488 1.31e-07 ***
I(mph^4)     1.255e-05  1.665e-06   7.539 1.17e-07 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.9307 on 23 degrees of freedom
Multiple R-squared:  0.9766,        Adjusted R-squared:  0.9726
F-statistic: 240.2 on 4 and 23 DF,  p-value: < 2.2e-16
```

Polynomials: example





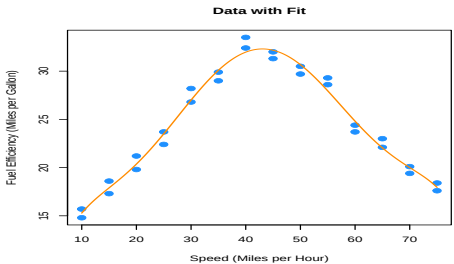
- The fourth order term is significant with the other terms in the model. Also we are starting to see what we expected for low and high speed. However, there still seems to be a bit of a pattern in the residuals, so we will again try more higher order terms.
- We will add the fifth and sixth together, since adding the fifth will be similar to adding the third.

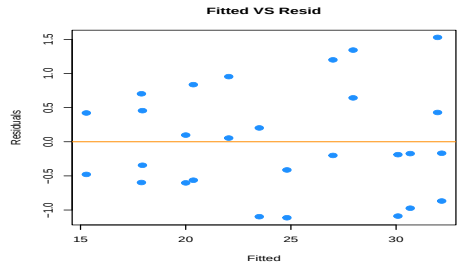
Polynomials: example

```
Call:
lm(formula = mpg ~ mph + I(mph^2) + I(mph^3) + I(mph^4) + I(mph^5) +
    I(mph^6), data = econ)

Residuals:
    Min      1Q  Median      3Q     Max
-1.1129 -0.5717 -0.1707  0.5026  1.5288

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept) -4.206e+00  1.204e+01  -0.349   0.7304
mph          4.203e+00  2.553e+00   1.646   0.1146
I(mph^2)    -3.521e-01  2.012e-01  -1.750   0.0947 .
I(mph^3)     1.579e-02  7.691e-03   2.053   0.0527 .
I(mph^4)    -3.473e-04  1.529e-04  -2.271   0.0338 *
I(mph^5)     3.585e-06  1.518e-06   2.362   0.0279 *
I(mph^6)    -1.402e-08  5.941e-09  -2.360   0.0280 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Polynomials: example

```
Residual standard error: 0.8657 on 21 degrees of freedom
Multiple R-squared:  0.9815,        Adjusted R-squared:  0.9762
F-statistic:   186 on 6 and 21 DF,  p-value: < 2.2e-16
```





Again the sixth order term is significant with the other terms in the model and here we see less pattern in the residuals plot.

Polynomials: example

- Let's now test for which of the previous two models we prefer. We will test

$$H_0 : \beta_5 = \beta_6 = 0.$$

```
Analysis of Variance Table

Model 1: mpg ~ mph + I(mph^2) + I(mph^3) + I(mph^4)
Model 2: mpg ~ mph + I(mph^2) + I(mph^3) + I(mph^4) + I(mph^5) + I(mph^6)
  Res.Df    RSS Df Sum of Sq      F Pr(>F)
1     23 19.922
2     21 15.739  2    4.1828 2.7905 0.0842 .
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

- This test does not reject the null hypothesis at a level of significance of $\alpha = 0.05$, however the p-value is still rather small, and the fitted versus residuals plot is much better for the model with the sixth order term. This makes the sixth order model a good choice.

Polynomials: example

- We could repeat this process one more time.

```
Analysis of Variance Table

Model 1: mpg ~ mph + I(mph^2) + I(mph^3) + I(mph^4) + I(mph^5) + I(mph^6)
Model 2: mpg ~ mph + I(mph^2) + I(mph^3) + I(mph^4) + I(mph^5) + I(mph^6) +
    I(mph^7) + I(mph^8)
  Res.Df    RSS Df Sum of Sq      F Pr(>F)
1     21 15.739
2     19 15.506  2    0.2324 0.1424 0.8682
```

  The eighth order term is not significant with the other terms in the model and the F-test does not reject.

- There is a quicker way to specify a model with many higher order terms. The method produces the same fitted values ...

```
fit6_alt <- lm(mpg ~ poly(mph, 6), data = econ)
all.equal(fitted(fit6), fitted(fit6_alt))
```

```
[1] TRUE
```

Polynomials: example

... but the estimated coefficients are different.

```
coef(fit6)
```

```
  (Intercept)           mph      I(mph^2)      I(mph^3)      I(mph^4)
-4.206224e+00  4.203382e+00 -3.521452e-01  1.579340e-02 -3.472665e-04
     I(mph^5)      I(mph^6)
 3.585201e-06 -1.401995e-08
```

```
coef(fit6_alt)
```

```
  (Intercept) poly(mph, 6)1 poly(mph, 6)2 poly(mph, 6)3 poly(mph, 6)4
  24.40714286    4.16769628  -27.66685755    0.13446747    7.01671480
poly(mph, 6)5 poly(mph, 6)6
   0.09288754   -2.04307796
```

- This is because `poly()` uses **orthogonal polynomials**.

Polynomials: example

- To use `poly()` to obtain the same results as using `I()` repeatedly, we would need to set `raw = TRUE`.

```
fit6_alt2 <- lm(mpg ~ poly(mph, 6, raw = TRUE), data = econ)
coef(fit6_alt2)
```

```
             (Intercept) poly(mph, 6, raw = TRUE)1 poly(mph, 6, raw = TRUE)2
           -4.206224e+00              4.203382e+00             -3.521452e-01
poly(mph, 6, raw = TRUE)3 poly(mph, 6, raw = TRUE)4 poly(mph, 6, raw = TRUE)5
            1.579340e-02             -3.472665e-04              3.585201e-06
poly(mph, 6, raw = TRUE)6
           -1.401995e-08
```

Melting artic

- Melting Arctic sea ice is monitored and used as an indicator for the impacts of climate change.
- The summer ice in the Arctic Ocean reflects sunlight. As the ice melts, the much darker sea water absorbs sunlight. This feedback mechanism is understood as an important driver of climate change throughout geologic history.
- Other effects: changes in ocean currents and atmospheric weather patterns as well as the possibility of releasing further greenhouse gases by accelerating the melting of Arctic permafrost on land and on the East Siberian Arctic Shelf.
- September is the month when the ice stops melting each summer and reaches its minimum extent.

Melting artic

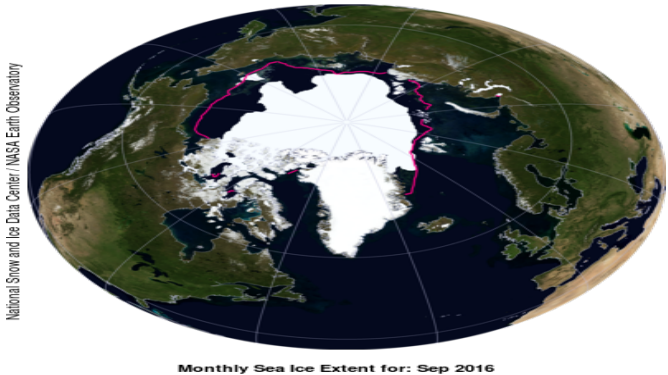
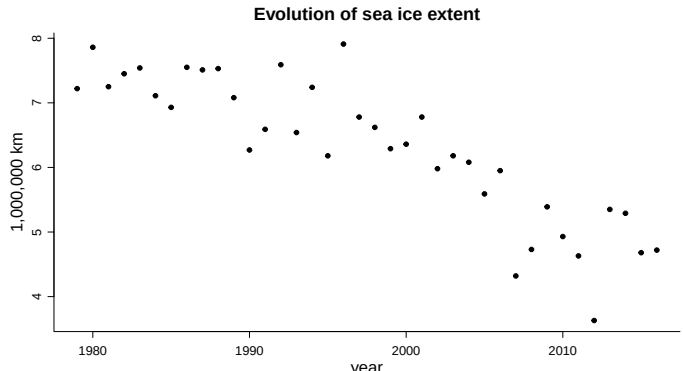


Figure: Source: National Snow and Ice Data Center

Melting artic

- The data we will analyze is a time series of September Arctic sea ice extent from 1979 until 2012.



A (simple) possible model

- Scientific question: "*Is the September Artic sea ice extent decreasing with time?*"

$$\text{Extent} = f(\text{Time})$$

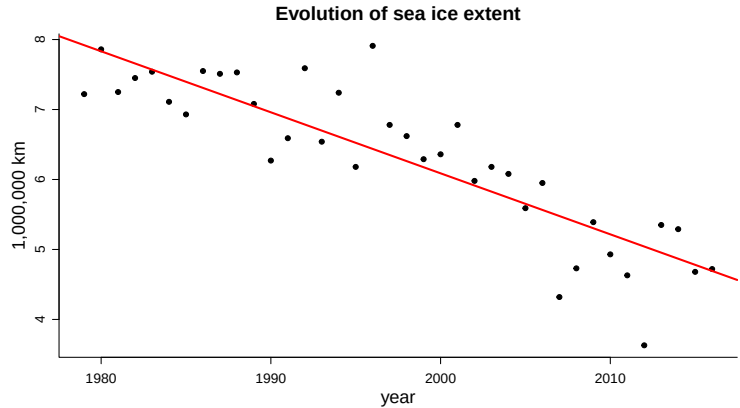
- Simple model: $f(\cdot)$ is **linear**

$$\text{Extent} = \beta_0 + \beta_1 \text{Time}$$

- Question: $\beta_1 < 0$?
- Linear regression model: n observations $y_1, y_2, \ldots, y_n$ from

$$Y_i = \beta_0 + \beta_1 x_i + \varepsilon_i, \quad (i = 1, \ldots, n)$$

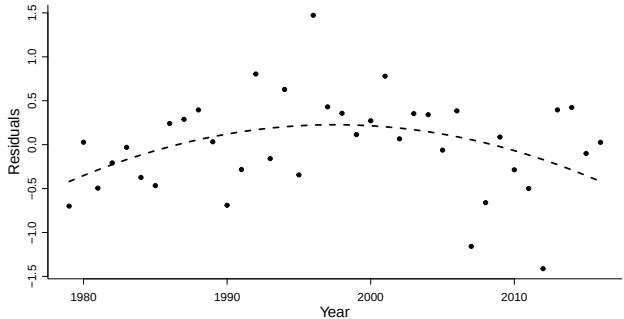
A (simple) possible model



- Fit is not too-bad, except for a few points...

A (simple) possible model

- The plot of residuals versus the time that reinforces this point.



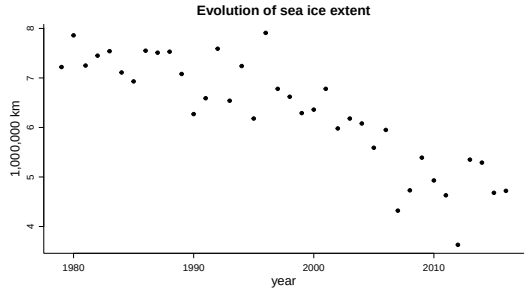
- The residuals from the linear regression tend to be negative in early and late years, and positive in the middle years.

A (simple) possible model

- Our previous model postulates ($x_i = t_i$)

$$\mathbb{E}[Y_i | t_i] = \beta_0 + \beta_1 t_i$$

- Look back at the data: the decrease is larger in the later years



A (simple) possible model

- The next table 1 shows the estimated slope from analogous regressions using data from 1979 until the end of September for each of the last sixteen years.
- Notice that the slope becomes steeper

starting.year	final.year	slope
1979	2001	-0.043
1979	2002	-0.049
1979	2003	-0.051
1979	2004	-0.053
1979	2005	-0.058
1979	2006	-0.059
1979	2007	-0.070
1979	2008	-0.077
1979	2009	-0.078
1979	2010	-0.080
1979	2011	-0.084
1979	2012	-0.091
1979	2013	-0.089
1979	2014	-0.087
1979	2015	-0.087
1979	2016	-0.087

A (simple) possible model

- Suppose that β_1 is changing with respect to time i.e. $\beta_1(t)$
- A simple model that allows for the slope to change at a constant rate.

$$\begin{aligned}
\mathbb{E}[Y_i|t_i] &= \beta_0 + \beta_1 t_i + \beta_2 t_i^2 \\
&= \beta_0 + \underbrace{\beta_1 \left[1 + \frac{\beta_2}{\beta_1} t_i\right]}_{\beta_1(t_i)} t_i \\
&= \beta_0 + \beta_1(t_i) t_i
\end{aligned}$$

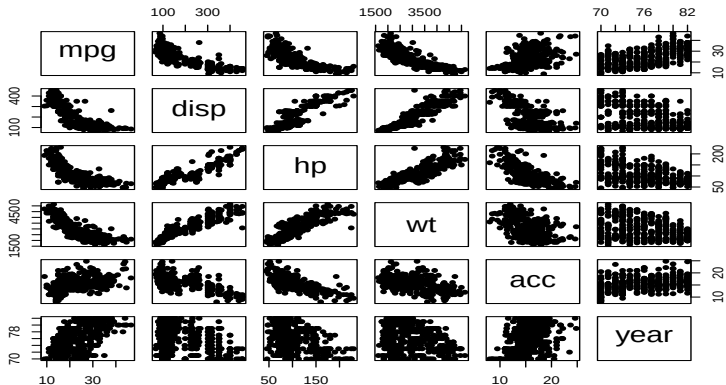
- A model with a quadratic term corresponds to a model in which the slope is allowed to change as a function of the predictor
- Linear model: the first derivative is constant. With quadratic terms: first derivative varies
- Similar concepts apply to higher order polynomials

Multicollinearity [6]

[6] Material in these slides was heavily influenced by Cosma Shalizi's notes.

Why collinearity is a problem

Let's look at the `autompg` dataset: many properties affect a car's efficiency:



Why collinearity is a problem

We could add all variables into the model:

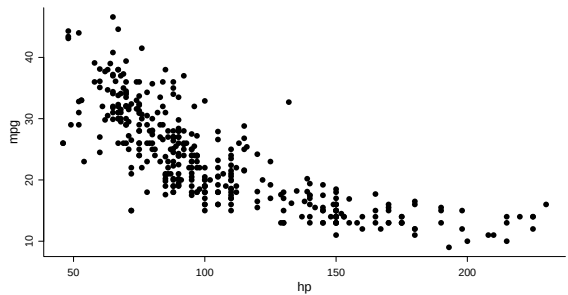
```
fit_big <- lm(mpg~., data = autompg); summary(fit_big)
```

```
Call:
lm(formula = mpg ~ ., data = autompg)

Residuals:
    Min      1Q  Median      3Q     Max
-8.3378 -2.3682 -0.1095  1.9052 14.2452

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept) -1.394e+01  4.606e+00  -3.026  0.00265
disp        -1.937e-03  5.544e-03  -0.349  0.72695
hp           6.274e-03  1.352e-02   0.464  0.64278
wt          -6.708e-03  6.619e-04 -10.134  < 2e-16
acc          2.796e-02  1.009e-01   0.277  0.78182
year         7.463e-01  5.189e-02  14.384  < 2e-16
```

Why collinearity is a problem

```
Residual standard error: 3.353 on 377 degrees of freedom
Multiple R-squared:  0.8198,        Adjusted R-squared:  0.8174
F-statistic:   343 on 5 and 377 DF,  p-value: < 2.2e-16
```

Model is significant! But.... β_{hp} is 0.00627416 and not significant.



Why collinearity is a problem

What is happening?

```
signif(cor(autompg),4)
```

```
        mpg    disp      hp      wt     acc    year
mpg   1.0000 -0.8176 -0.7800 -0.8424  0.4224  0.5786
disp -0.8176  1.0000  0.9027  0.9352 -0.5632 -0.3720
hp   -0.7800  0.9027  1.0000  0.8687 -0.6947 -0.4154
wt   -0.8424  0.9352  0.8687  1.0000 -0.4337 -0.3127
acc   0.4224 -0.5632 -0.6947 -0.4337  1.0000  0.2922
year  0.5786 -0.3720 -0.4154 -0.3127  0.2922  1.0000
```

Strong relationships to `mpg` but lots of correlation between variables

Why collinearity is a problem

- The estimated coefficients in a multiple linear regression is

$$\widehat{\beta} = (X^\top X)^{-1} X^\top y$$

  provided that $X^\top X$ is invertible.
- Similarly, the variance of the estimates,

$$\mathrm{Var}\left[\widehat{\beta}\right] = \sigma^2 (X^\top X)^{-1}$$

  will blow up when $X^\top X$ is singular.
- If that matrix isn't exactly singular, but is close to being non-invertible, the estimation will become unstable and the variances will become huge.
- The estimation becomes unreliable and very variable
- (Might not be a big problem if all we care about is prediction)

Why collinearity is a problem

- It looks like we have p different predictor variables (**Important**: I've included as predictor variable the column defining the constant term!), but really some of them are linear combinations of the others, so they don't add any information.
- The real number of distinct variables is $q < p$, the column rank of X.
- If the exact linear relationship holds among more than two variables, we talk about **multicollinearity**.
- **Collinearity** can refer either to the general situation of a linear dependence among the predictors, or, by contrast to multicollinearity, a linear relationship among just two of the predictors.

Dealing with collinearity by deleting predictor variables

- Since not all of the p predictor variables are actually contributing information, a natural way of dealing with collinearity is to drop some predictor variables from the model.
- If you want to do this, you should think very carefully about *which* predictor variable to delete.
- Ideally you would use knowledge on the measuring processes and about the process under study

Diagnosing collinearity among pairs of predictor variables

- Easy: we make the pairs plot of all the predictor variables, and we see if any of them fall on a straight line, or close to one.
- If the number of predictor variables *is huge*, look at the correlation matrix, and worry about any entry off the diagonal which is (nearly) ± 1.
- But a multicollinear relationship involving three or more predictor variables might be totally invisible on a pairs plot.
- We use some checks and metrics which can help identify issues

Diagnosing collinearity among pairs of predictor variables

- Red flag 1: Large changes in estimated coefficients when one other predictor variable is included or removed
- Red flag 2: non-significant results in individual tests on β_j for variables X_j which appear to be important when taken individually
- Red flag 3: estimated value of β_j with opposite sign from what we see in scatterplot of (X_j, Y) or that we expect from theoretical considerations
- Red flag 4: large sample correlations between X_is and X_js.
- Is $(X^\top X)$ (almost) singular: check if any eigenvalues are ≈ 0?
- If any eigenvalue is exactly 0 some of the β_j can not be estimated
- A more formal quantity that we can compute is the Variance Inflation Factor (VIF)

Variance inflation factors (VIF)

- If the predictors were uncorrelated, the variance of $\hat{\beta}_i$ would be

$$\mathrm{Var}\left[\hat{\beta}_i\right] = \frac{\sigma^2}{ns^2_{X_i}}$$

- With correlated predictors we have

$$\mathrm{Var}\left[\hat{\beta}_i\right] = \sigma^2 (X^\top X)^{-1}_{i+1,i+1}$$

- We define the the **variance inflation factor** for the i^{th} coefficient the ratio of the two variances:

$$\mathrm{VIF}_i = (X^\top X)^{-1}_{i+1,i+1} * ns^2_{X_i}$$

- The average of the variance inflation factors across all predictors is often noted $\overline{\mathrm{VIF}}$, or just VIF.

Variance inflation factors (VIF)

- It is possible to show that variance inflation factor for X_i can be found by regressing X_i on all of the other X_j, computing the R^2 of this regression say R_i^2, and setting

$$\text{VIF}_i = \frac{1}{1 - R_i^2}$$

- Consequence: if $\text{VIF}_i \geq 1$, i.e. the predictors are correlated with each other, the standard errors of the coefficient estimates will be bigger than if the predictors were uncorrelated.
- The variance inflation factor increases as X_i becomes more correlated with some linear combination of the other predictors ($\text{VIF}_i \geq 10 \Leftrightarrow R_i^2 \geq 0.9$)
- Folklore says that $\text{VIF}_i > 10$ indicates "serious" multicollinearity for the predictor.
- Folklore also says that a $VIF >> 1$ indicates "serious" multicollinearity.
- These are rules of thumb, not hard boundaries.

Variance inflation factors (VIF)

For the `autompg` model we have:

```
car::vif(fit_big)
```

```
    disp       hp       wt      acc     year
11.492407  9.357184 10.894270  2.631376  1.240917
```

We try to drop `disp` and `wt`:

```
car::vif(lm(mpg ~ hp + acc + year, data = autompg))
```

```
      hp      acc     year
2.136251 1.932649 1.208566
```

Variance inflation factors (VIF)

```
summary(lm(mpg ~ hp + acc + year, data = autompg))
```

```
Call:
lm(formula = mpg ~ hp + acc + year, data = autompg)

Residuals:
     Min       1Q   Median       3Q      Max
-11.7092  -2.8513  -0.6334   2.1798  15.5216

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)  1.006697   5.636870   0.179    0.858
hp          -0.164426   0.008104 -20.289  < 2e-16
acc         -0.662701   0.108492  -6.108  2.5e-09
year         0.657771   0.064254  10.237  < 2e-16

Residual standard error: 4.208 on 379 degrees of freedom
Multiple R-squared:  0.7148,        Adjusted R-squared:  0.7125
F-statistic: 316.6 on 3 and 379 DF,  p-value: < 2.2e-16
```

β_{hp} now is negative and significant.

Is multicollinearitya problem?

Multicollinearity is another instance of the model correctness vs. usefulness.

- A model with multicollinearity might be perfectly valid in the sense of respecting the assumptions of the model. It does not matter whether the predictors are related or not. At least for the verification of the assumptions.
- But the model will be useless if the multicollinearity is high, since it can inflate the variability of the estimation without any kind of bound.

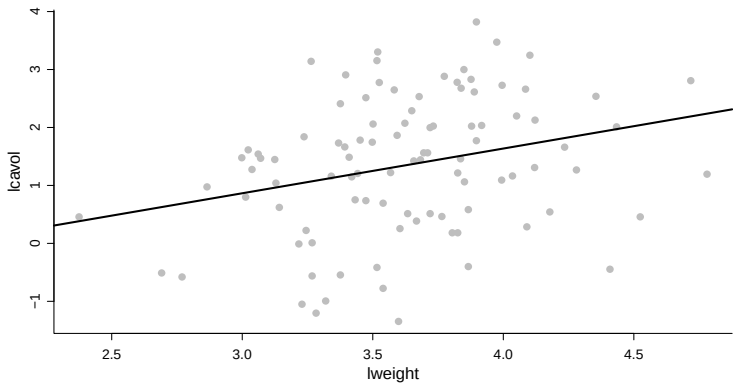
Some more advanced methods do exist to deal with multicollinearity, for example *ridge regression* or *Priciple Components Regression*: we do not discuss these in the course.

Influential points [7]

[7] Material in based on the Lecture 20 of Cosma Shalizi's course

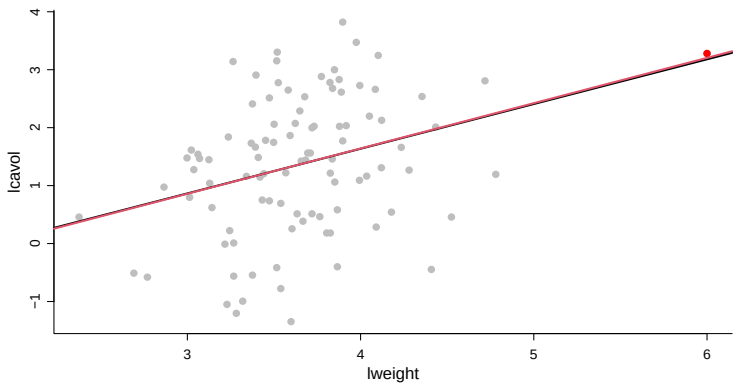
Influential points

The `prostate` dataset: focus on the relationship between `lcavol` and `lweight`



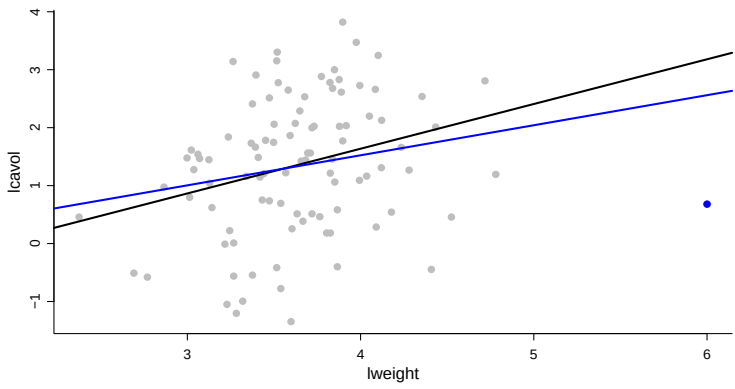
Influential points

Imagine now we had some additional observations:



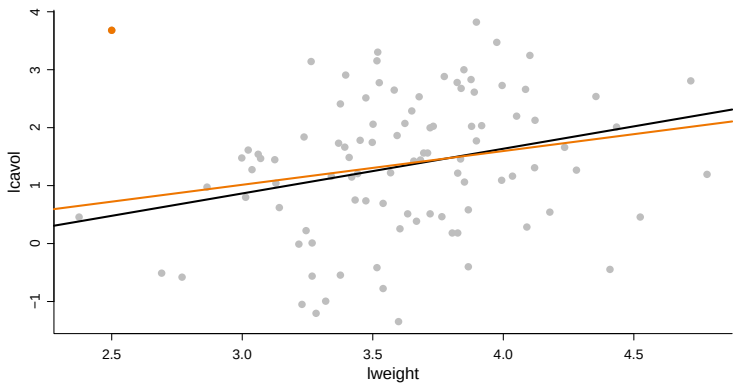
Influential points

Imagine now we had some additional observations:



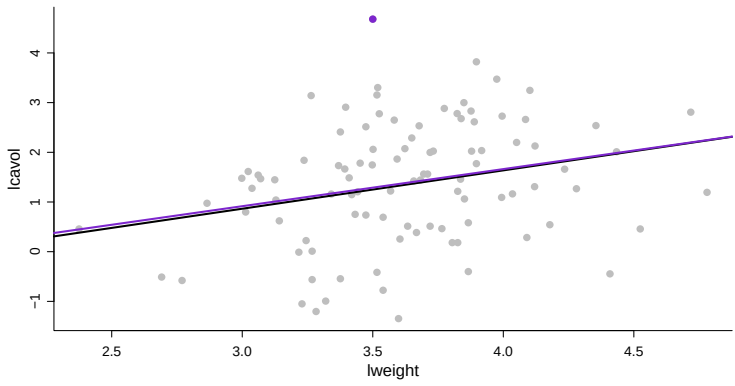
Influential points

Imagine now we had some additional observations:



Influential points

Imagine now we had some additional observations:



Influential points

- Some points can be particularly **influential** in the estimation of (β_0, β_1).
- These are points which break the pattern of the main relationship between X and Y.
- Their values might be not pattern-breaking when considering the x_i value or y_i value, but the combination of the (x_i, y_i) value might be at odds with the rest of the data.
- On the other hand points which have extreme values of X and Y but which fall in line with the rest of the data do not greatly affect the estimation of (β_0, β_1).
- We want to have a way to quantify if any individual point is unduly affecting our estimation
- Let's see what happens in the simple linear regression case

Influential points

- In SLR we know that
$$\hat{\beta}_0 = \overline{y} - \hat{\beta}_1 \overline{x}$$
- Let us turn this around. The fitted value at $X = \overline{x}$ is
$$\hat{\beta}_0 + \hat{\beta}_1 \overline{x} = \overline{y}$$
- Suppose we had a data point, say the i^{th} point, where $X = \overline{x}$. Then the actual value of y_i almost wouldn't matter for the fitted value there — the regression line *has* to go through $\overline{y}$ at $\overline{x}$, never mind whether y_i there is close to $\overline{y}$ or far away.
- If $x_i = \overline{x}$, we say that y_i has little *leverage* over $\hat{m}_i$, or little *influence* on $\hat{m}_i$.
- It has *some* influence, because y_i is part of what we average to get $\overline{y}$, but that's not a lot of influence.

Influential points

- Moreover, with simple linear regression, we know that

$$\hat{\beta}_1 = \frac{c_{XY}}{s_X^2}$$

- How does y_i show up in this? It's

$$\hat{\beta}_1 = \frac{n^{-1}\sum_{i=1}^{n}(x_i - \overline{x})(y_i - \overline{y})}{s_X^2}$$

- Notice that when $x_i = \overline{x}$, y_i doesn't actually matter at all to the slope.
- If x_i is far from $\overline{x}$, then $y_i - \overline{y}$ will contribute to the slope, and its contribution will get bigger (whether positive or negative) as $x_i - \overline{x}$ grows.
- y_i will also make a big contribution to the slope when $y_i - \overline{y}$ is big (unless, again, $x_i = \overline{x}$).

Influential points

- Let's write a general formula for the predicted value, at an arbitrary point $X = x$.

$$\begin{aligned}
\hat{m}(x) &= \hat{\beta}_0 + \hat{\beta}_1 x \\
&= \overline{y} - \hat{\beta}_1 \overline{x} + \hat{\beta}_1 x \\
&= \overline{y} + \hat{\beta}_1 (x - \overline{x}) \\
&= \overline{y} + \frac{1}{n} \frac{\sum_{i=1}^{n} (x_i - \overline{x})(y_i - \overline{y})}{s_X^2} (x - \overline{x})
\end{aligned}$$

So, in words:

- The predicted value is always a weighted average of all the y_i.
- As x_i moves away from $\overline{x}$, y_i gets more weight (possibly a large negative weight). When $x_i = \overline{x}$, y_i only matters because it contributes to the global mean $\overline{y}$.
- The weights on all data points increase in magnitude when the point x where we're trying to predict is far from $\overline{x}$. If $x = \overline{x}$, only $\overline{y}$ matters.

Influential points

- All of this is still true of the fitted values at the original data points:
  - If x_i is at $\overline{x}$, y_i only matters for the fit because it contributes to $\overline{y}$.
  - As x_i moves away from $\overline{x}$, in either direction, it makes a bigger contribution to *all* the fitted values.
- Why is this happening? We get the coefficient estimates by minimizing the mean squared error, and the MSE treats all data points equally:

$$\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{m}(x_i))^2$$

- But we're not just using any old function $\hat{m}(x)$; we're using a linear function.
- This has only two parameters, so we can't change the predicted value to match each data point — altering the parameters to bring $\hat{m}(x_i)$ closer to y_i might actually increase the error elsewhere.

Influential points

- By minimizing the over-all MSE with a linear function, we get two constraints,

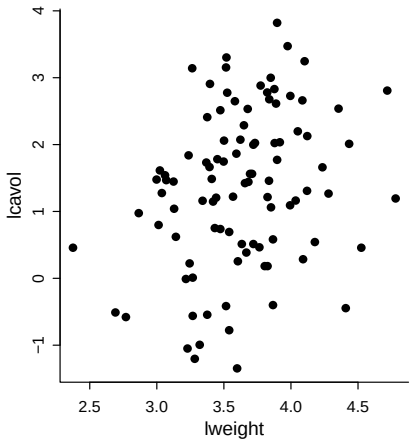
$$\overline{y} = \hat{\beta}_0 + \hat{\beta}_1 \overline{x} \quad \text{and} \quad \sum_i e_i (x_i - \overline{x}) = 0$$

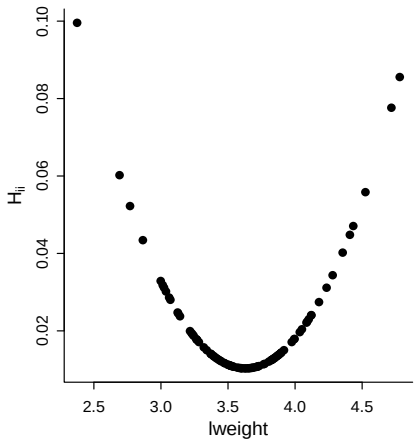
- The first of these makes the regression line insensitive to y_i values when x_i is close to $\overline{x}$.
- The second makes the regression line *very* sensitive to residuals when $(x_i - \overline{x})$ is big. When $(x_i - \overline{x})$ is large, a big residual is harder to balance out than if $(x_i - \overline{x})$ were smaller.
- So, let's sum this up:
  - Least squares estimation tries to bring all the predicted values closer to y_i, but it can't match each data point at once, because the fitted values are all functions of the same coefficients.
  - If x_i is close to $\overline{x}$, y_i makes little difference to the coefficients or fitted values — they're pinned down by needing to go through the mean of the data.
  - As x_i moves away from $\overline{x}$, $y_i - \overline{y}$ makes a bigger and bigger impact on both the coefficients and on the fitted values.

Influential points

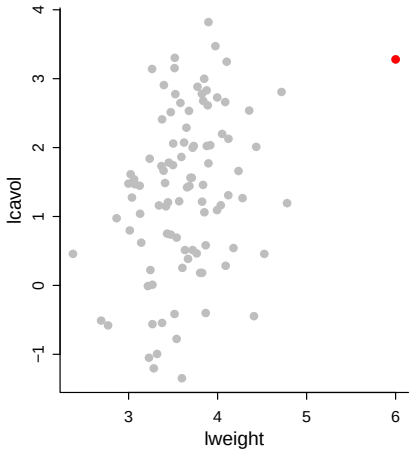
- Points which don't fall on the same regression line as the others, might throw off our estimate
- This is going to be a concern when x_i is far from $\overline{x}$, or when the combination of $x_i - \overline{x}$ and $y_i - \overline{y}$ makes that point have a disproportionate impact on the estimates.
- We also worry about points with large residuals, particularly when they correspond to values with large $(x_1 - \overline{x})$ value: the model tries very hard to fit some individual points
- All of this also holds for multiple regression models where things become more complicated because of the high dimensionality (harder to know when x_i is *far* from $\overline{x}$) and we need to deal with matrices)

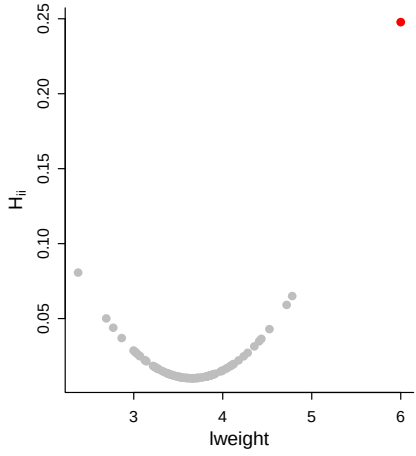
Leverage


lcavol
lweight
2.5
3.0
3.5
4.0
4.5
−1
0
1
2
3
4

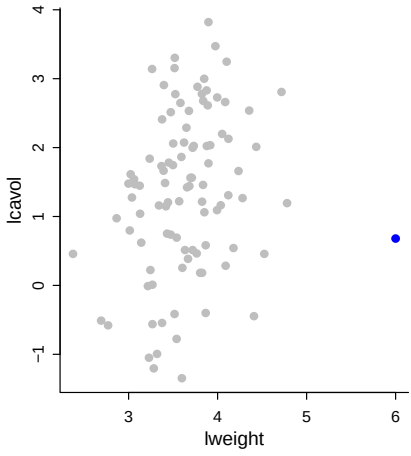

H_{ii}
lweight
0.02
0.04
0.06
0.08
0.10
2.5
3.0
3.5
4.0
4.5

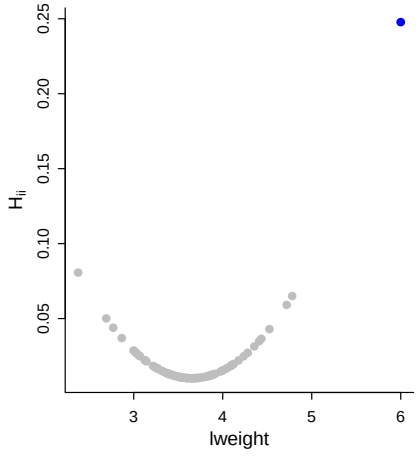
Leverage


lcavol
lweight
3
4
5
6
−1
0
1
2
3
4

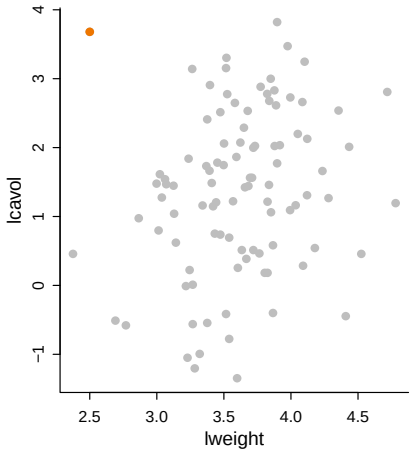

H_{ii}
lweight
0.05
0.10
0.15
0.20
0.25
3
4
5
6

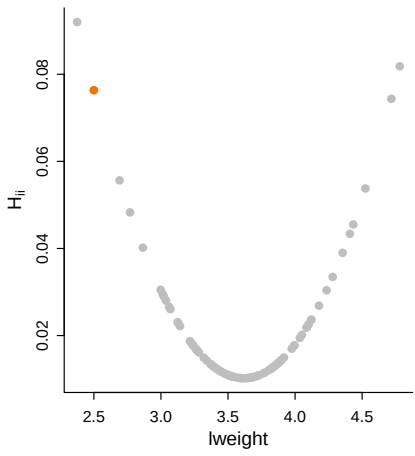
Leverage


lcavol
lweight
4
3
2
1
0
−1
3
4
5
6

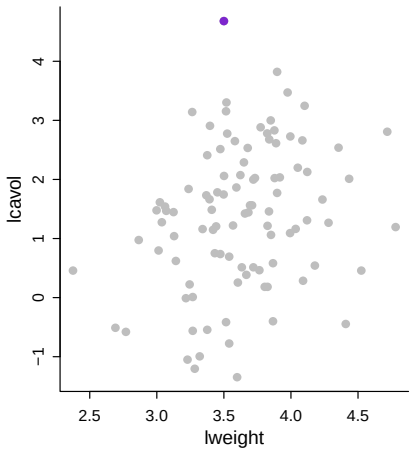

H_{ii}
lweight
0.25
0.20
0.15
0.10
0.05
3
4
5
6

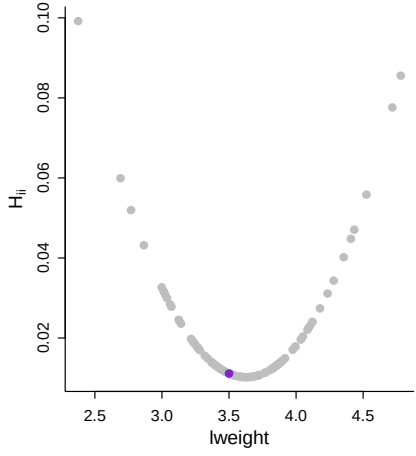
Leverage


lcavol
lweight
2.5
3.0
3.5
4.0
4.5
−1
0
1
2
3
4


H_{ii}
lweight
0.02
0.04
0.06
0.08
2.5
3.0
3.5
4.0
4.5

Leverage


lcavol
lweight
2.5
3.0
3.5
4.0
4.5
−1
0
1
2
3
4


H_{ii}
lweight
0.02
0.04
0.06
0.08
0.10
2.5
3.0
3.5
4.0
4.5

Standardized and Studentized residuals

Let's consider the model residuals:

$$\mathbf{e} = \mathbf{y} - \widehat{\mathbf{m}} = (\mathbf{I} - \mathbf{H})\mathbf{y} = (\mathbf{I} - \mathbf{H})\epsilon$$

We then have that:

$$\mathbb{E}[\mathbf{e}] = \mathbf{0} \text{ and } \mathrm{Var}[\mathbf{e}] = \sigma^2(\mathbf{I} - \mathbf{H})(\mathbf{I} - \mathbf{H})^T = \sigma^2(\mathbf{I} - \mathbf{H})$$

The variance of the residual for the i^{th} data point depends on i through the hat matrix:

$$\mathrm{Var}[e_i] = \sigma^2(\mathbf{I} - \mathbf{H})_{ii} = \sigma^2(1 - H_{ii})$$

So the bigger the leverage of i, the smaller the variance of the residual there (the model tries very hard to fit these points).
We define the **standardized** or (internally) **studentized residuals**

$$r_i \equiv \frac{e_i}{\hat{\sigma}\sqrt{1 - H_{ii}}}$$

Standardized and Studentized residuals

Why "studentized"? Because we're dividing by an estimate of the standard error, just like in "Student's" t-test for differences in means

Standardized residuals are sometimes preferred in residual plots, as they have been standardized to have equal variance.

Indeed the plots we get when applying `plot` to an `lm` object in R are based on the Standardized residuals, which can be obtained using `rstandard`

Standardized and Studentized residuals

In defining the Leave-one-out cross validation (LOOCV) we considered what would be our estimate of y_i if y_i was not included in the dataset.

We denoted this value with with $\hat{y}_{[i]}$ while $e_{[i]}$ as the residual for the ith observation, when that observation is not used to fit the model:

$$e_{[i]} = y_i - \hat{y}_{[i]}$$

Leaving out the data point i would give us an MSE of $\hat{\sigma}^2_{[i]}$. A little work says that

$$t_i \equiv \frac{e_{[i]}}{\hat{\sigma}_{[i]}\sqrt{1 + \mathbf{x}_i^T(\mathbf{x}_{[i]}^T\mathbf{x}_{[i]}^{-1})\mathbf{x}_i}} \sim t_{n-p}$$

These are called the **cross-validated**, or **jackknife**, or **externally studentized**, residuals.

Standardized and Studentized residuals

Fortunately, we can compute this without having to actually re-run the regression:

$$
\begin{aligned}
t_i &= \frac{e_{[i]}}{\hat{\sigma}_{[i]}\sqrt{1+\mathbf{x}_i^T(\mathbf{x}_{[i]}^T\mathbf{x}_{[i]})^{-1}\mathbf{x}_i}} \\
&= \frac{e_i}{\hat{\sigma}_{[i]}\sqrt{1-H_{ii}}} \\
&= r_i\sqrt{\frac{n-p}{n-p-r_i^2}}
\end{aligned}
$$

Cook's Distance

Omitting point i will generally change all of the fitted values, not just the fitted value at that point: is the change big for a certain point i?
Compare $\widehat{\mathbf{m}}$ and $\widehat{\mathbf{m}}_{[i]}$. Specifically we use a squared difference:

$$\|\widehat{\mathbf{m}} - \widehat{\mathbf{m}}_{[i]}\|^2 = (\widehat{\mathbf{m}} - \widehat{\mathbf{m}}_{[i]})^T(\widehat{\mathbf{m}} - \widehat{\mathbf{m}}_{[i]})$$

To make this more comparable across data sets, it's conventional to divide this by $p\ \hat{\sigma}^2$.

This is called the **Cook's distance** or **Cook's statistic** for point i:

$$D_i = \frac{(\widehat{\mathbf{m}} - \widehat{\mathbf{m}}_{[i]})^T(\widehat{\mathbf{m}} - \widehat{\mathbf{m}}_{[i]})}{(p)\hat{\sigma}^2}$$

Cook's Distance

As usual, there is a simplified formula, which evades having to re-fit the regression:

$$D_i = \frac{1}{p} e_i^2 \frac{H_{ii}}{(1 - H_{ii})^2}$$

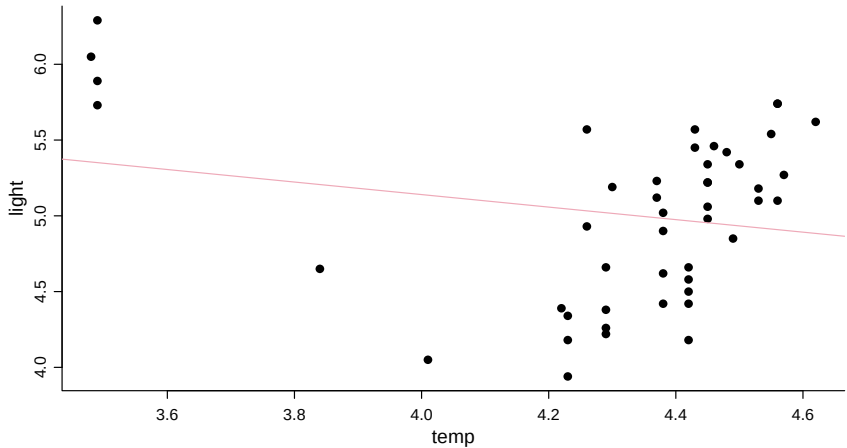
The total influence of a point over all the fitted values grows with both its leverage (H_{ii}) and the size of its residual when it is included (e_i^2).

CYG OB1 stars

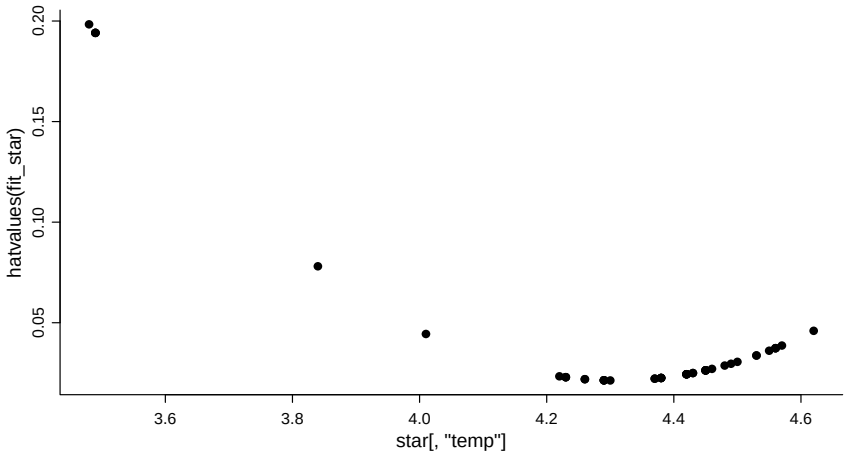
The data `stars` contains information on the log of the surface temperature and the log of the light intensity of 47 stars in the star cluster CYG OB1, which is in the direction of Cygnus.

```
data(star, package = "faraway")
plot(star)
fit_star <- lm(light ~ temp, data = star)
```

CYG OB1 stars

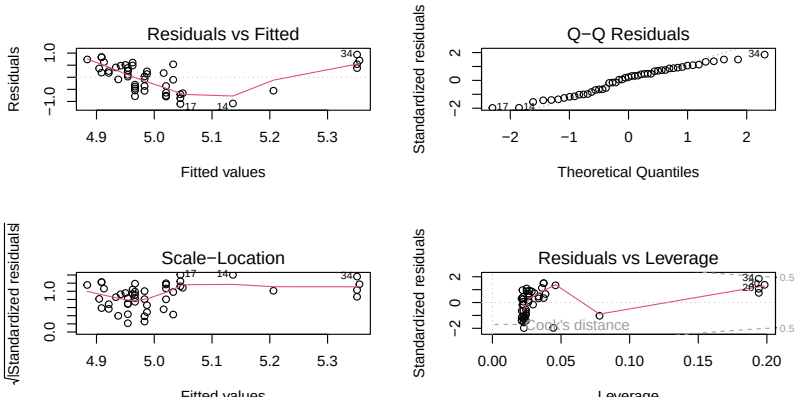


CYG OB1 stars



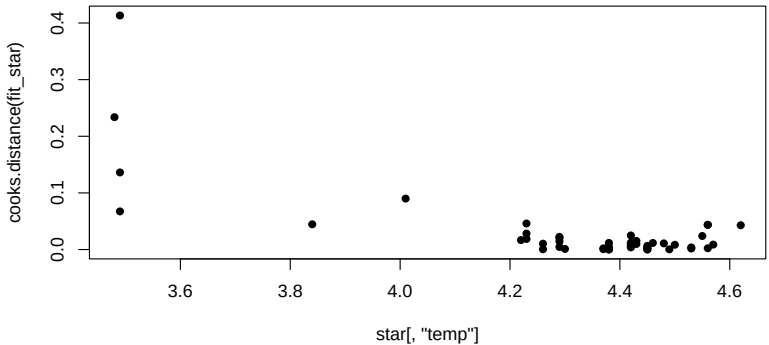
CYG OB1 stars

```
par(mfrow=c(2,2)); plot(fit_star)
```



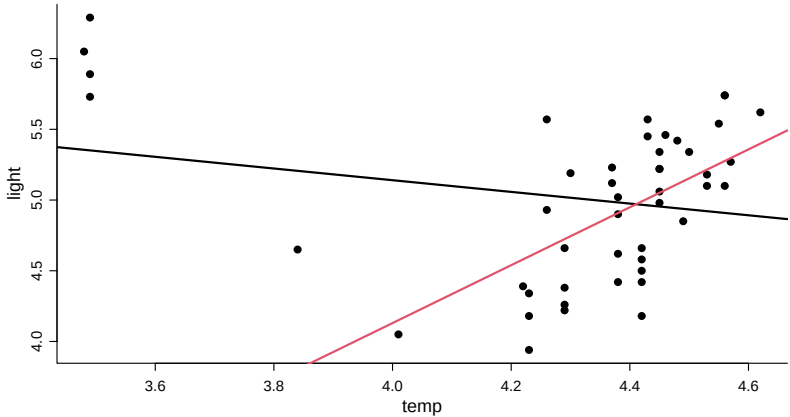
CYG OB1 stars

```
plot(star[,"temp"], cooks.distance(fit_star), pch = 16)
```



CYG OB1 stars

Fitted lines with and without giant stars:



What to do about influential points

- Automatic detection of influential points can be useful to find points which are problematic
- Sometimes these methods help us identify data recording issues or problematic measurement and we can discard the points
- But sometime they are the symptom of something more complicated going on in the process: need to question what is the origin of these outlying points

Table of content